
Verbindung von Virtual Shared Spaces über weite Entfernungen Entwurf



Erstellt von:

Karlsruhe
26.06.2018

Inhaltsverzeichnis

1	Einleitung	4
2	Aufbau	5
2.1	Architektur	5
2.2	Klassendiagramm	7
	Klassen Hierarchie	8
3	Paket GUI	9
3.1	Klasse GUIManager	10
4	Paket PTV.Vissim.Interfaces	12
4.1	Klasse ExchangeData	13
4.2	Klasse Simulator_Ped_Data	14
4.3	Klasse Simulator_Veh_Data	15
4.4	Klasse VissimInterface	16
4.5	Klasse VISSIMPedestrianData	19
4.6	Klasse VISSIMSignalData	21
4.7	Klasse VISSIMVehicleData	22
5	Paket UnityEngine.CoreModule	24
5.1	Klasse Behaviour	26
5.2	Klasse Camera	28
5.3	Klasse Car	29
5.4	Klasse CarUserControl	30
5.5	Klasse Component	32
5.6	Klasse FirstPersonController	34
5.7	Klasse GameObject	36
5.8	Klasse HideFlags	38
5.9	Klasse MessageBase	39
5.10	Klasse MonoBehaviour	40
5.11	Klasse NetworkTransform	42
5.12	Klasse NetworkView	44
5.13	Klasse Object	45
5.14	Klasse Pedestrian	47
5.15	Klasse Quaternion	48
5.16	Klasse Rigidbody	49
5.17	Klasse Scene	50
5.18	Klasse Transform	51
5.19	Klasse Vector3	52
6	Paket UnityEngine.Networking	53
6.1	Klasse CustomManager	55
6.2	Klasse NetworkBehaviour	58
6.3	Klasse NetworkClient	60
6.4	Klasse NetworkConnection	61
6.5	Klasse NetworkIdentity	62
6.6	Klasse NetworkManager	64
6.7	Klasse NetworkManagerHUD	67

6.8	Klasse NetworkServer	68
6.9	Klasse PlayerController	69
7	Abläufe	70
7.1	Sequenzdiagramme	70
8	Anhang	75

Abbildungsverzeichnis

1	Simulationsumgebung	4
2	Architekturmodell	5
3	Klassendiagramm komplett	7
4	Klassendiagramm - GUI Paket	9
5	Klassendiagramm - Paket PTV.Vissim.Interfaces	12
6	Klassendiagramm - Paket UnityEngine.CoreModule	24
7	Klassendiagramm - Paket UnityEngine.Networking	53
8	Sequenz-Diagramm 1 : Host erstellt Server	71
9	Sequenz-Diagramm 2 : Gast startet die Simulation und verbindet sich falsch	73

1 Einleitung

Bei diesem Dokument handelt es sich um das Entwurfsdokument zur Architektur des Projektes „Virtual Shared Spaces über weite Entfernungen“. Dabei geht es um die Verwirklichung einer Netzwerkschnittstelle zu einer bereits vorhandenen Simulationsumgebung, welche mithilfe der Tools Vissim und Unity ermöglicht wurde.

In diesem Dokument wird das Klassendiagramm näher vorgestellt und auf die einzelnen im Diagramm verwendeten Pakete näher eingegangen. Jedes Paket besteht aus einer Vielzahl an Klassen. Im Anschluss folgt zu jeder Klasse eine Beschreibung, welche die Funktionsweise, Methoden und Attribute näher erklärt. Zusätzlich sind verschiedene Sequenzdiagramme vorhanden, welche sich auf Szenarien aus dem Pflichtenheft beziehen und den zeitlichen Ablauf typischer Programmvorgänge zeigen.

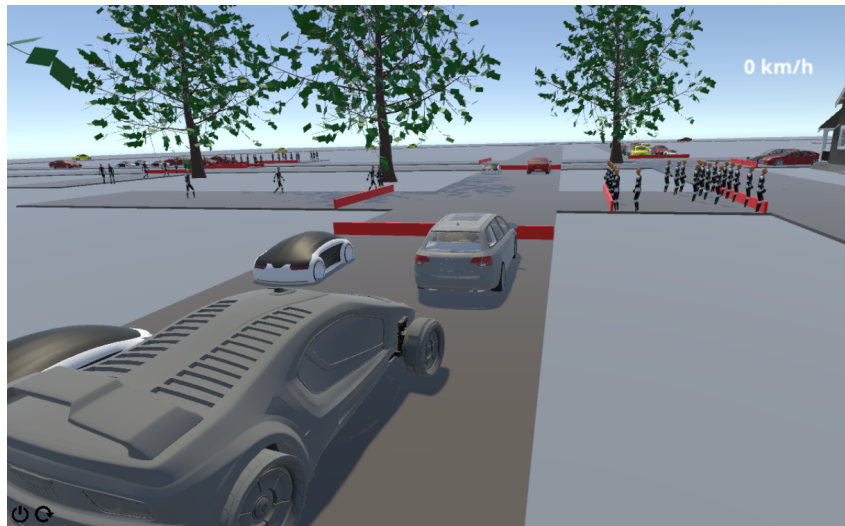


Abbildung 1: Simulationsumgebung

2 Aufbau

2.1 Architektur

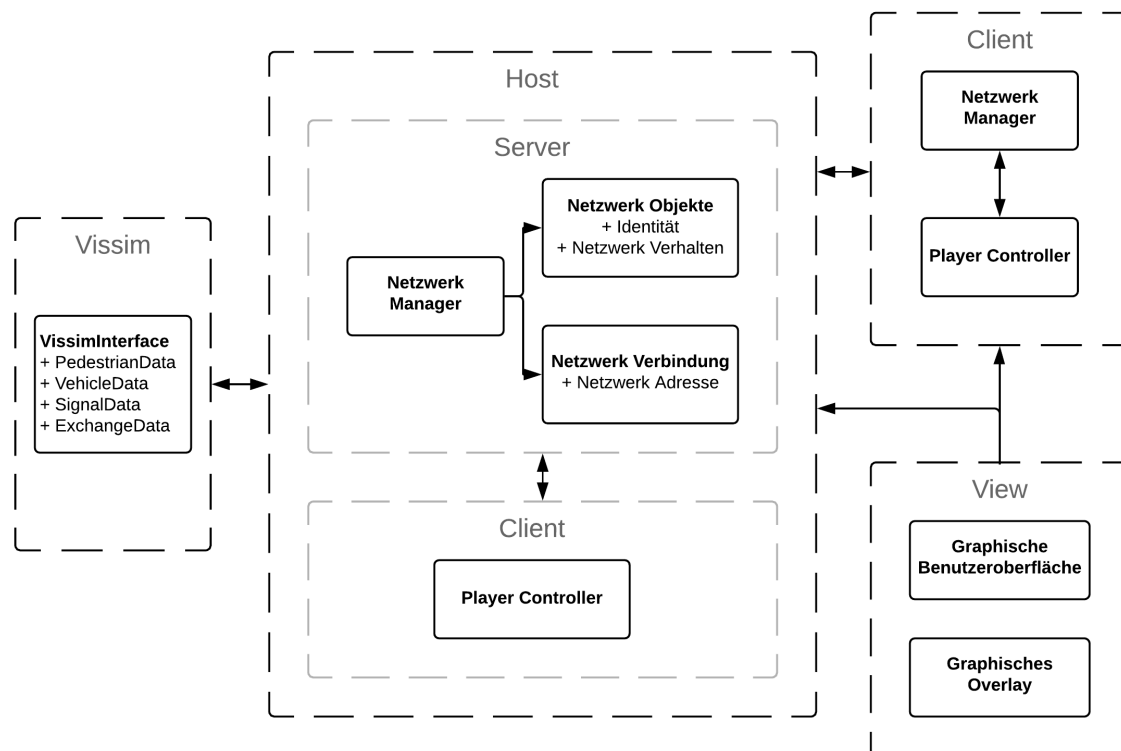


Abbildung 2: Architekturmodell

Die Grundstruktur des Programmes ist in vier Teile aufgeteilt. Die Verteilung in View, Host, Client und Vissim soll sowohl eine leichtere Modifikation des Programmes, als auch einer Erleichterung der Erweiterbarkeit dienen. Die vier Komponenten erfüllen folgende Aufgaben:

2.1.1 View

Die View ermöglicht dem Benutzer des Programmes das Starten der Simulation durch bereitstellen einer graphischen Benutzeroberfläche. Die Oberfläche beinhaltet eine Ansicht für das Verbinden als Gast sowie das Erstellen eines Servers als Host. Während der Simulation ist die View für das Darstellen eines graphischen Overlays verantwortlich. Dieses dient dazu dem Benutzer Informationen wie seine IP-Adresse und die Anzahl der auf der Simulationsumgebung verbundenen Clients anzuzeigen.

2.1.2 Client

Der Client ist das Programm welches auf dem Endsystem des Benutzers ausgeführt wird, um sich mit dem Server der Simulation zu verbinden. Er besteht aus den Bestandteilen Netzwerk-Manager und Player-Controller. Ersteres ist für die Verbindung, Kommunikation und Synchronisation mit dem Server verantwortlich. Der Player-Controller handhabt die Bewegung des Benutzermodells in der Umgebung. Dafür verwendet er die Eingaben des Benutzers.

2.1.3 Host

Der Host besteht aus einem Server und einem Client. Der Server hat einen Netzwerk-Manager, welcher sich um die Synchronisation der Netzwerkobjekte über das Netzwerk kümmert. Außerdem handhabt er die verbundenen Clients und ist damit verantwortlich für die Netzwerkverbindung, welche von ihm bereitgestellt wird. Die Client Komponente des Hosts beinhaltet einen Player-Controller, damit ist der Host in der Lage auf dem eigens von ihm bereitgestellten Server eine Verbindung aufzunehmen und selbst ein Benutzermodell in der Simulationsumgebung zu bewegen.

2.1.4 Vissim

Vissim ist ein eigenständiges Programm, welches über eine Schnittstelle mit dem Host kommuniziert. Es stellt während der Laufzeit der Simulation Positionsdaten der Fußgänger und Autos, welche Computergesteuert in der Umgebung bewegt werden, zur Verfügung.

2.2 Klassendiagramm

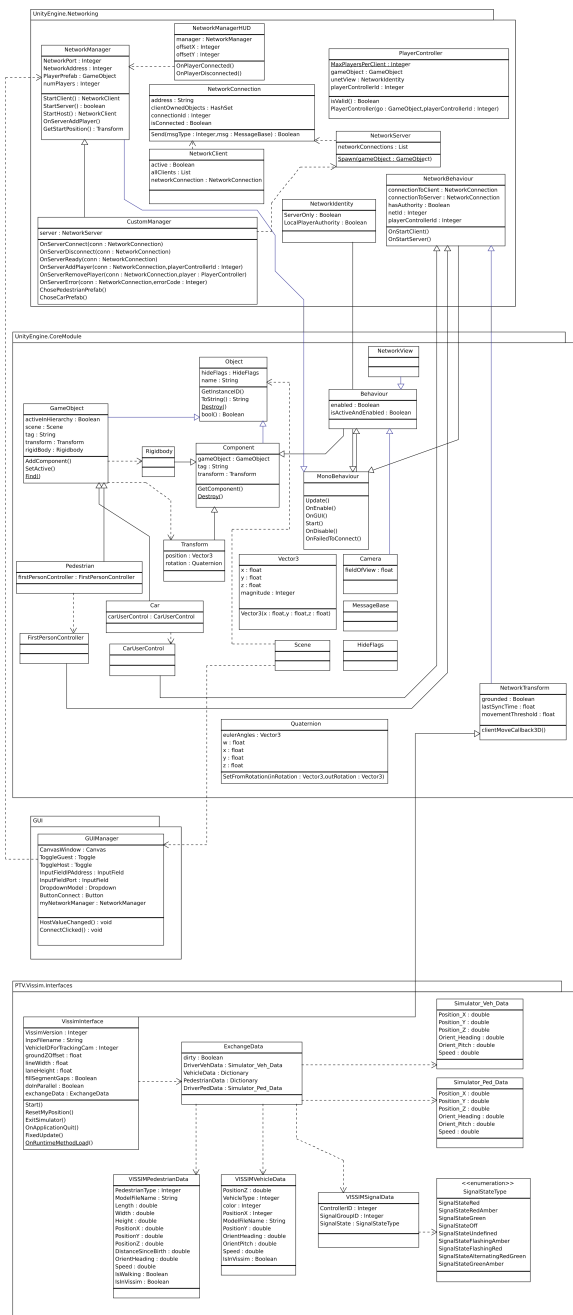


Abbildung 3: Klassendiagramm komplett

Klassen Hierarchie

Die Klassen Hierarchie spiegelt die strukturelle und verhaltensbezogene Ähnlichkeit der Klassen wieder. Dabei erbt die jeweils untere Klassen von der Elternklasse, der sogenannten Basisklasse.

Klassen

- UnityEngine.CoreModule.Object (in 5.13, Seite 45)
 - GUI.GUIManager (in 3.1, Seite 10)
 - PTV.Vissim.Interfaces.ExchangeData (in 4.1, Seite 13)
 - PTV.Vissim.Interfaces.Simulator_Ped_Data (in 4.2, Seite 14)
 - PTV.Vissim.Interfaces.Simulator_Veh_Data (in 4.3, Seite 15)
 - PTV.Vissim.Interfaces.VISSIMPedestrianData (in 4.5, Seite 19)
 - PTV.Vissim.Interfaces.VISSIMVehicleData (in 4.7, Seite 22)
 - PTV.Vissim.Interfaces.VissimSignalData
 - UnityEngine.CoreModule.HideFlags (in 5.8, Seite 38)
 - UnityEngine.CoreModule.MessageBase (in 5.9, Seite 39)
 - UnityEngine.CoreModule.Object (in 5.13, Seite 45)
 - UnityEngine.CoreModule.Component (in 5.5, Seite 32)
 - UnityEngine.CoreModule.Behaviour (in 5.1, Seite 26)
 - UnityEngine.CoreModule.Camera (in 5.2, Seite 28)
 - UnityEngine.CoreModule.MonoBehaviour (in 5.10, Seite 40)
 - UnityEngine.Networking.NetworkBehaviour (in 6.2, Seite 58)
 - UnityEngine.CoreModule.CarUserControl (in 5.4, Seite 30)
 - UnityEngine.CoreModule.FirstPersonController (in 5.6, Seite 34)
 - UnityEngine.CoreModule.NetworkTransform (in 5.11, Seite 42)
 - PTV.Vissim.Interfaces.VISSIMInterface
 - UnityEngine.Networking.NetworkIdentity (in 6.5, Seite 62)
 - UnityEngine.Networking.NetworkManager (in 6.6, Seite 64)
 - UnityEngine.Networking.CustomManager (in 6.1, Seite 55)
 - UnityEngine.CoreModule.NetworkView (in 5.12, Seite 44)
 - UnityEngine.CoreModule.Rigidbody (in 5.16, Seite 49)
 - UnityEngine.CoreModule.Transform (in 5.18, Seite 51)
 - UnityEngine.CoreModule.GameObject (in 5.7, Seite 36)
 - UnityEngine.CoreModule.Car (in 5.3, Seite 29)
 - UnityEngine.CoreModule.Pedestrian (in 5.14, Seite 47)
 - UnityEngine.CoreModule.Quaternion (in 5.15, Seite 48)
 - UnityEngine.CoreModule.Scene (in 5.17, Seite 50)
 - UnityEngine.CoreModule.Vector3 (in 5.19, Seite 52)
 - UnityEngine.Networking.NetworkClient (in 6.3, Seite 60)
 - UnityEngine.Networking.NetworkConnection (in 6.4, Seite 61)
 - UnityEngine.Networking.NetworkManagerHUD (in 6.7, Seite 67)
 - UnityEngine.Networking.NetworkServer (in 6.8, Seite 68)
 - UnityEngine.Networking.PlayerController (in 6.9, Seite 69)

3 Paket GUI

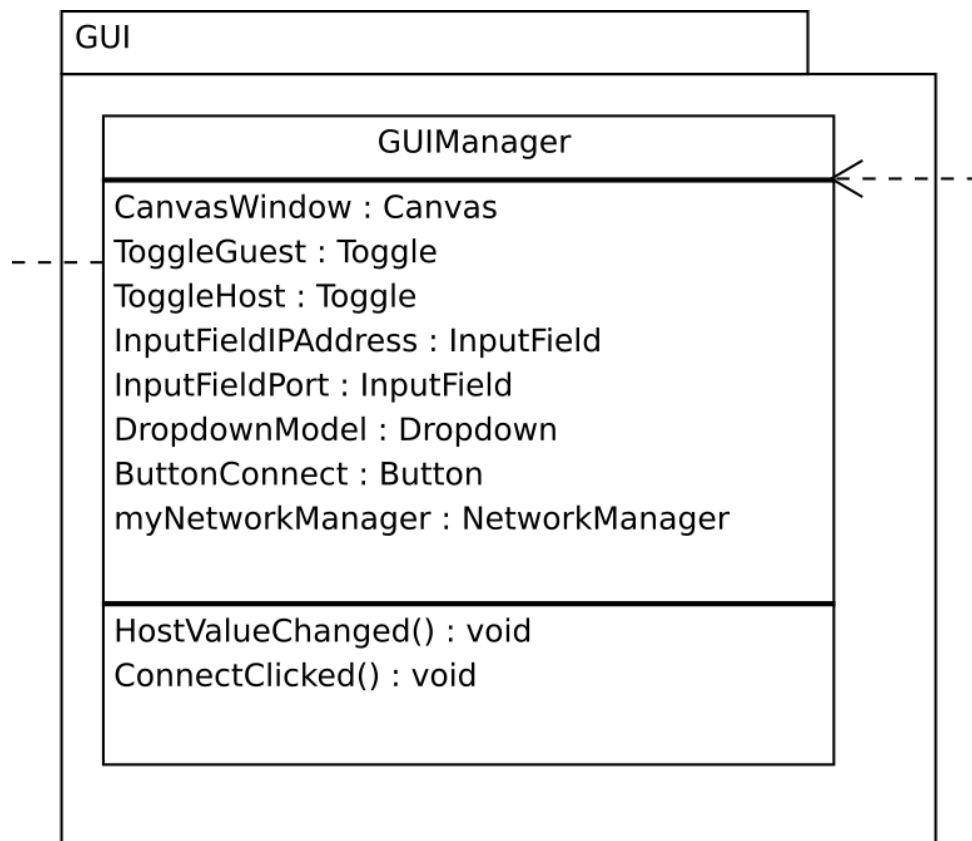


Abbildung 4: Klassendiagramm - GUI Paket

Beschreibung

Das GUI Paket beinhaltet die Klassen der graphischen Benutzeroberfläche, welche für das Verbinden zu einem Host und für das Erstellen eines Host Servers verantwortlich sind.

Paket Inhalt

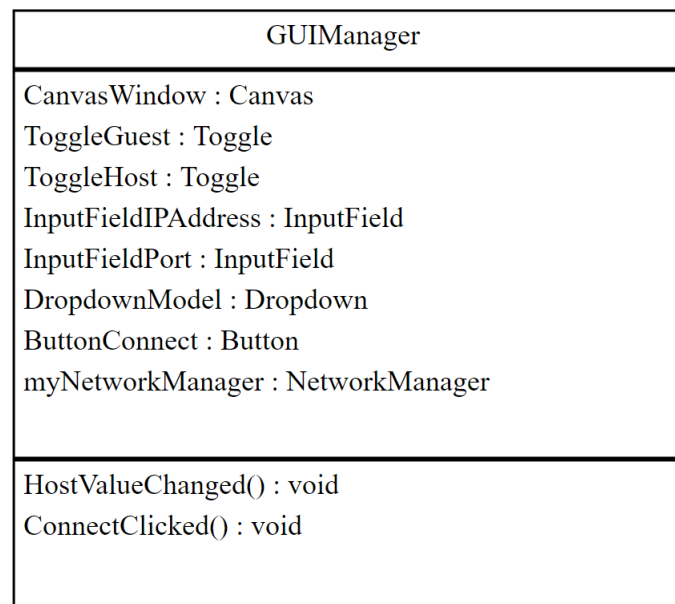
Seite

Klassen

GUIManager 10

Diese Klasse baut die graphische Benutzeroberfläche auf und reagiert auf die Benutzung der Elemente.

3.1 Klasse GUIManager



Diese Klasse baut die graphische Benutzeroberfläche auf und reagiert auf die Benutzung der Elemente.

3.1.1 Deklaration

```
public class GUIManager
    erweitert UnityEngine.CoreModule.Object
```

3.1.2 Attribute

- **public UnityEngine.UIModule.Canvas CanvasWindow**
 - Element, in dem die Elemente der Benutzeroberfläche angeordnet und geladen werden.
- **public UnityEngine.UI.Toggle ToggleGuest**
 - Toggle zur Auswahl der Guest-Option.
- **public UnityEngine.UI.Toggle ToggleHost**
 - Toggle zur Auswahl der Host-Option.
- **public UnityEngine.UI.InputField InputFieldIPAddress**
 - Eingabefeld zur Eingabe der Host-IPAdresse.
- **public UnityEngine.UI.InputField InputFieldPort**
 - Eingabefeld zur Auswahl des zu verwendenden Ports.
- **public UnityEngine.UI.Dropdown DropdownModel**
 - Aufklappbare Liste zur Auswahl des Modells.
- **public UnityEngine.UI.Button ButtonConnect**
 - Knopf zum Verbinden mit einem durch die Eingabefelder ausgewählten Host oder zum Starten eines Servers.
- **public UnityEngine.Networking.NetworkManager myNetworkManager**
 - Die semantische Verbindung zwischen der graphischen Benutzeroberfläche und der Netzwerkverbindung. Durch den Netzwerk-Manager wird hier zum Beispiel geprüft,

ob eine Verbindung aufgebaut werden kann. Diese Information wird dann entsprechend in der Benutzeroberfläche angezeigt.

3.1.3 Konstruktoren

- **GUIManager**
`public GUIManager()`

3.1.4 Methoden

- **HostValueChanged**
`public void HostValueChanged()`
 - **Beschreibung**
Wird aufgerufen, wenn sich der Status der Toggles ändert. Verändert die graphische Benutzerfläche entsprechend der Änderung.
- **ConnectClicked**
`public void ConnectClicked()`
 - **Beschreibung**
Wird aufgerufen, wenn der Connect-Button gedrückt wurde. Überprüft, welcher Toggle ausgewählt ist und versucht eine Verbindung mit dem durch die Eingabefelder ausgewählten Host aufzubauen oder startet einen Server.

4 Paket PTV.Vissim.Interfaces

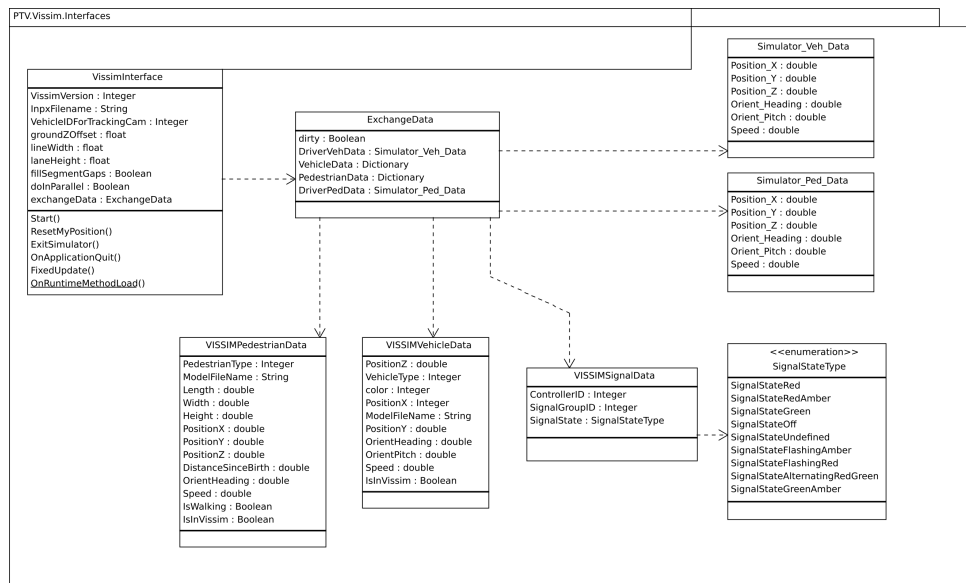


Abbildung 5: Klassendiagramm - Paket PTV.Vissim.Interfaces

Beschreibung

PTV Vissim ist ein eigenständiges Programm welches mit Unity interagiert um Fußgänger und Fahrzeuge innerhalb der Welt zu bewegen. Die hier dargestellten Klassen stellen die für das Projekt wichtigen Schnittstellen, welche zur Kommunikation und Synchronisation der Simulation dienen, dar.

Paket Inhalt

Seite

Klassen

ExchangeData	13
Eine Klasse die zum Datenaustausch zwischen der 3D-Simulation und der Vissimsimulation genutzt wird.	
Simulator_Ped_Data	14
Die allgemeinen Daten eines Fußgängers in der Unity 3D-Simulation.	
Simulator_Veh_Data	15
Die allgemeinen Daten eines Fahrzeugs in der Unity 3D-Simulation.	
VISSIMInterface	??
Die VISSIMInterface Klasse ist das Hauptinterface, dass die Kommunikation zwischen dem 3D-Unity Simulator und dem PTV-Vissim Simulator möglich macht.	
VISSIMPedestrianData	19
Die allgemeinen Daten eines Fußgängers in der Vissimsimulation.	
VissimSignalData	??
Die allgemeinen Daten einer Ampelschaltung in der Vissimsimulation.	
VISSIMVehicleData	22
Die allgemeinen Informationen eines Fahrzeugs in der Vissimsimulation.	

4.1 Klasse ExchangeData

ExchangeData
dirty : Boolean DriverVehData : Simulator_Veh_Data VehicleData : Dictionary PedestrianData : Dictionary DriverPedData : Simulator_Ped_Data

Eine Klasse die zum Datenaustausch zwischen der 3D-Simulation und der Vissimsimulation genutzt wird.

4.1.1 Deklaration

```
public class ExchangeData
    erweitert UnityEngine.CoreModule.Object
```

4.1.2 Attribute

- **public bool dirty**
 - Ist immer wahr, wenn die Daten der lokalen Simulation nicht mit der Vissimsimulation übereinstimmen.
- **public Simulator_Veh_Data DriverVehData**
 - Die Daten der Fahrzeuge in der 3D-Simulation.
- **public System.Collections.Generic.Dictionary<int, VISSIMVehicleData> VehicleData**
 - Die Fahrzeuge aus der Vissimsimulation.
- **public System.Collections.Generic.Dictionary<int, VISSIMPedestrianData> PedestrianData**
 - Die Fußgänger aus der Vissimsimulation.
- **public Simulator_Ped_Data DriverPedData**
 - Die Daten der Fußgänger in der 3D-Simulation.

4.1.3 Konstruktoren

- **ExchangeData**

```
public ExchangeData()
```

4.2 Klasse Simulator_Ped_Data

Simulator_Ped_Data
Position_X : double Position_Y : double Position_Z : double Orient_Heading : double Orient_Pitch : double Speed : double

Die allgemeinen Daten eines Fußgängers in der Unity 3D-Simulation.

4.2.1 Deklaration

```
public class Simulator_Ped_Data
    erweitert UnityEngine.CoreModule.Object
```

4.2.2 Attribute

- **public double Position_X**
 - Die X-Koordinate der Fußgängerposition.
- **public double Position_Y**
 - Die Y-Koordinate der Fußgängerposition.
- **public double Position_Z**
 - Die Z-Koordinate der Fußgängerposition.
- **public double Orient_Heading**
 - Maß für die Euler Drehung bei Bewegung, im Bogenmaß.
- **public double Orient_Pitch**
 - Maß für die Euler Drehung bei Bewegung, im Bogenmaß.
- **public double Speed**
 - Die Geschwindigkeit des Fußgängers in Meter pro Sekunde.

4.2.3 Konstruktoren

- **Simulator_Ped_Data**

```
public Simulator_Ped_Data()
```

4.3 Klasse Simulator_Veh_Data

Simulator_Veh_Data
Position_X : double Position_Y : double Position_Z : double Orient_Heading : double Orient_Pitch : double Speed : double

Die allgemeinen Daten eines Fahrzeugs in der Unity 3D-Simulation.

4.3.1 Deklaration

```
public class Simulator_Veh_Data
    erweitert UnityEngine.CoreModule.Object
```

4.3.2 Attribute

- **public double Position_X**
– Die X-Koordinate der Fahrzeugposition.
- **public double Position_Y**
– Die Y-Koordinate der Fahrzeugposition.
- **public double Position_Z**
– Die Z-Koordinate der Fahrzeugposition.
- **public double Orient_Heading**
– Maß für die Euler Drehung bei Bewegung, im Bogenmaß.
- **public double Orient_Pitch**
– Maß für die Euler Drehung bei Bewegung, im Bogenmaß.
- **public double Speed**
– Die Geschwindigkeit des Fahrzeugs in Meter pro Sekunde.

4.3.3 Konstruktoren

- **Simulator_Veh_Data**
`public Simulator_Veh_Data()`

4.4 Klasse VissimInterface

VissimInterface
VissimVersion : Integer InpxFilename : String VehicleIDForTrackingCam : Integer groundZOffset : float lineWidth : float laneHeight : float fillSegmentGaps : Boolean doInParallel : Boolean exchangeData : ExchangeData
Start() ResetMyPosition() ExitSimulator() OnApplicationQuit() FixedUpdate() <u>OnRuntimeMethodLoad()</u>

Die VissimInterface Klasse ist das Hauptinterface, dass die Kommunikation zwischen dem 3D-Unity Simulator und dem PTV-Vissim Simulator möglich macht.

4.4.1 Deklaration

```
public class VissimInterface
    erweitert UnityEngine.CoreModule.NetworkTransform
```

4.4.2 Attribute

- **public int VissimVersion**
– Die Vissim Versionsnummer.
- **public string InpxFilename**
– Der Name der Datei, welche benutzt wird um die Simulationsszene zu erstellen.
- **public int VehicleIDForTrackingCam**
- **public float groundZOffset**
– Der Offset auf der Z-Koordinate der Szene.
- **public float laneHeight**
– Die Fahrbahnhöhe, welche für den Import der VISSIM eigenen .inpx Dateien verwendet wird.
- **public bool fillSegmentGaps**
– Sollte wahr sein, wenn die Lücken zwischen den Straßensegmenten ausgefüllt werden sollen.
- **public bool doInParallel**
– Ist immer wahr, wenn die Kommunikation mit der Vissimsimulation parallel stattfinden soll.
- **public ExchangeData exchangeData**

- Die Daten, die zwischen der 3D-Simulation und der Vissimsimulation ausgetauscht werden.

4.4.3 Konstruktoren

- **VISSIMInterface**
public VissimInterface()

4.4.4 Methoden

- **ExitSimulator**
public void ExitSimulator()
 - **Beschreibung**
Beim Aufruf dieser Methode wird die Simulation verlassen.
- **FixedUpdate**
public void FixedUpdate()
 - **Beschreibung**
Eine Aktualisierungsmethode, die kontinuierlich während der Simulation aufgerufen wird, um diese zu aktualisieren.
- **OnApplicationQuit**
public void OnApplicationQuit()
 - **Beschreibung**
Beendet die Verbindung der VISSIM Schnittstelle mit VISSIM.
- **OnRuntimeMethodLoad**
public static void OnRuntimeMethodLoad()
 - **Beschreibung**
Erlaubt die Initialisierung von Klassenmethoden ohne Eingaben eines Benutzers.
- **ResetMyPosition**
public void ResetMyPosition()
 - **Beschreibung**
Setzt die Position des Benutzers auf einen Standardwert zurück.
- **Start**
public void Start()
 - **Beschreibung**
Initialisiert die Vissimsimulation und erstellt die Simulationsszene.

4.4.5 Attribute geerbt von Klasse NetworkTransform

UnityEngine.CoreModule.NetworkTransform (in [5.11](#), Seite [42](#))

- public void clientMoveCallback3D()
- public bool grounded
- public float lastSyncTime
- public float movementThreshold
- public void newOperation()

4.4.6 Attribute geerbt von Klasse NetworkBehaviour

UnityEngine.Networking.NetworkBehaviour (in 6.2, Seite 58)

- public NetworkConnection connectionToClient
- public NetworkConnection connectionToServer
- public bool hasAuthority
- public int netId
- public void OnStartClient()
- public void OnStartServer()
- public int playerControllerId

4.4.7 Attribute geerbt von Klasse MonoBehaviour

UnityEngine.CoreModule.MonoBehaviour (in 5.10, Seite 40)

- public void OnDisable()
- public void OnEnable()
- public void OnFailedToConnect()
- public void OnGUI()
- public void Start()
- public void Update()

4.4.8 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in 5.1, Seite 26)

- public bool enabled
- public bool isActiveAndEnabled

4.4.9 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in 5.5, Seite 32)

- public static void Destroy()
- public GameObject gameObject
- public void GetComponent()
- public string tag
- public Transform transform

4.4.10 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in 5.13, Seite 45)

- public bool bool()
- public static void Destroy()
- public void GetInstanceID()
- public HideFlags hideFlags
- public string name
- public string ToString()

4.5 Klasse VISSIMPedestrianData

VISSIMPedestrianData
PedestrianType : Integer ModelFileName : String Length : double Width : double Height : double PositionX : double PositionY : double PositionZ : double DistanceSinceBirth : double OrientHeading : double Speed : double IsWalking : Boolean IsInVissim : Boolean

Die allgemeinen Daten eines Fußgängers in der Vissimsimulation.

4.5.1 Deklaration

```
public class VISSIMPedestrianData
    erweitert UnityEngine.CoreModule.Object
```

4.5.2 Attribute

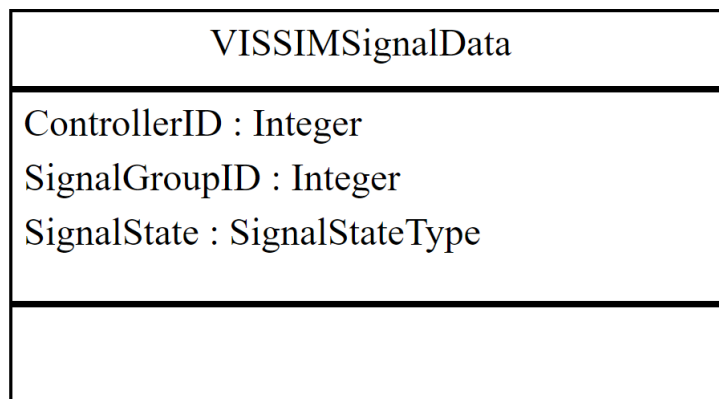
- **public int PedestrianType**
 - Die Fußgängertypnummer der Vissimsimulation.
- **public string ModelFileName**
 - Der Name der Datei in der das Fußgängermodel gespeichert ist.
- **public double Length**
 - Die Länge des Fußgänger Models in Metern.
- **public double Width**
 - Die Breite des Fußgänger Models in Metern.
- **public double Height**
 - Die Höhe des Fußgänger Models in Metern.
- **public double PositionX**
 - Die X-Koordinate der Fußgängerposition.
- **public double PositionY**
 - Die Y-Koordinate der Fußgängerposition.
- **public double PositionZ**
 - Die Z-Koordinate der Fußgängerposition.
- **public double DistanceSinceBirth**
 - Die Distanz, die der Fußgänger seit seiner Erstellung gelaufen ist, in Metern.

- **public double OrientHeading**
 - Die Blickrichtung der Fußgänger, im Bogenmaß.
- **public double Speed**
 - Die Geschwindigkeit des Fußgängers in Metern pro Sekunde.
- **public bool IsWalking**
 - Ist immer wahr, wenn der Fußgänger läuft.
- **public bool IsInVissim**
 - Ist immer wahr, wenn der Fußgänger sich gerade in der Simulation befindet.

4.5.3 Konstruktoren

- **VISSIMPedestrianData**
`public VISSIMPedestrianData()`

4.6 Klasse VISSIMSignalData



Die allgemeinen Daten einer Ampelschaltung in der Vissimsimulation.

4.6.1 Deklaration

```
public class VISSIMSignalData
    erweitert UnityEngine.CoreModule.Object
```

4.6.2 Attribute

- public int **ControllerID**
 - Die ID-Nummer des Controllers.
- public int **SignalGroupID**
 - Die ID-Nummer der Ampelgruppe.
- public SignalStateType **SignalState**
 - Der aktuelle Zustand der Ampel.

4.6.3 Konstruktoren

- **VISSIMSignalData**

```
public VISSIMSignalData()
```

4.7 Klasse VISSIMVehicleData

VISSIMVehicleData
PositionZ : double VehicleType : Integer color : Integer PositionX : Integer ModelFileName : String PositionY : double OrientHeading : double OrientPitch : double Speed : double IsInVissim : Boolean

Die allgemeinen Informationen eines Fahrzeugs in der Vissimsimulation.

4.7.1 Deklaration

```
public class VISSIMVehicleData
    erweitert UnityEngine.CoreModule.Object
```

4.7.2 Attribute

- **public double PositionZ**
– Die Z-Koordinate der Fahrzeugposition.
- **public int VehicleType**
– Die Fahrzeugtypnummer der Vissimsimulation.
- **public int color**
– Die Farbe des Fahrzeugs im RGB-Format.
- **public double PositionX**
– Die X-Koordinate der Fahrzeugposition.
- **public string ModelFileName**
– Der Name der Datei, in der das Fahrzeugmodel gespeichert ist.
- **public double PositionY**
– Die Y-Koordinate der Fahrzeugposition.
- **public double OrientHeading**
– Die Blickrichtung des Autos, im Bogenmaß.
- **public double OrientPitch**
– Die Neigung des Autos, im Bogenmaß.
- **public double Speed**

- Die Geschwindigkeit des Fahrzeugs in Metern pro Sekunde.
- `public bool IsInVissim`
 - Ist immer wahr, wenn das Fahrzeug sich gerade in der Simulation befindet.

4.7.3 Konstruktoren

- `VISSIMVehicleData`
 - `public VISSIMVehicleData()`

5 Paket UnityEngine.CoreModule

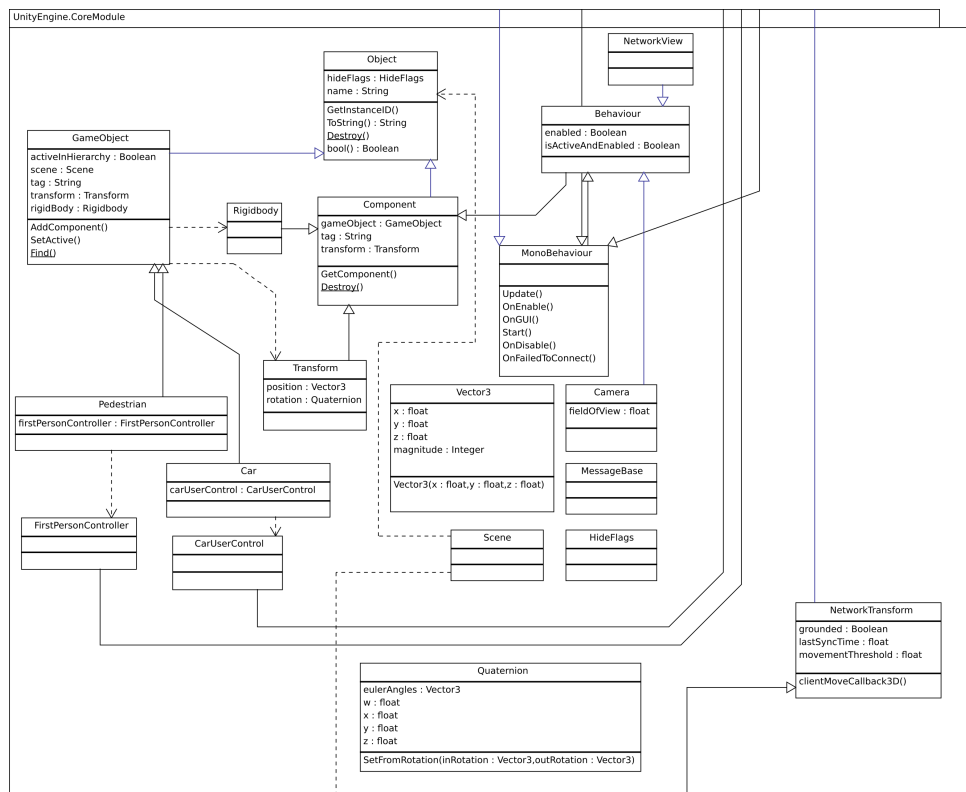


Abbildung 6: Klassendiagramm - Paket UnityEngine.CoreModule

Beschreibung

Das UnityEngine.CoreModule Paket beinhaltet die für das Projekt wichtigen Unity Klassen, welche zur Umsetzung benötigt werden. Diese Klassen sind bereits von Unity vorgegeben und werden für die Realisierung des Projektes angepasst.

Paket Inhalt

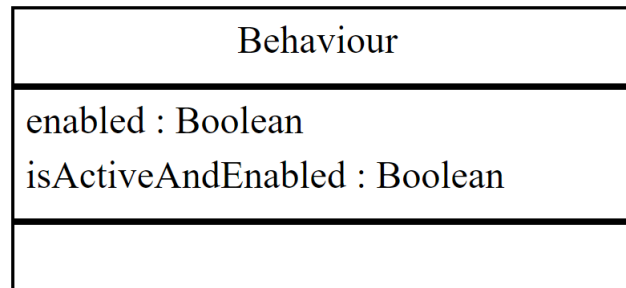
Seite

Klassen

Behaviour	26
Dies ist die Oberklasse aller Behaviour Objekte. Behaviour Objekte sind Komponente welche an Objekte gehängt werden um dem Server mitzuteilen, wie dieses Objekt in der Simulation zu verwalten ist.	
.....	28
Die Klasse Camera definiert die Sicht, welche ein Benutzer in der Simulation hat.	
Car	29
Das Autofahrer Objekt welches ein Benutzer auswählen kann, um dieses in der Simulation zu verwenden.	
CarUserController	30
Diese Klasse steuert das Benutzer Model Auto.	
Component	32
Die Oberklasse für alle Objektverknüpfungen.	

FirstPersonController	34
Diese Klasse steuert das Model des Benutzers aus einer First-Person Ansicht.	
GameObject	36
Diese Klasse ist die Oberklasse aller Entitäten/Instanzen die in Unity Szenen genutzt werden.	
HideFlags	38
Gibt an ob das Objekt versteckt ist, mit der Szene gespeichert oder vom Benutzer modifizierbar sein soll.	
MessageBase	39
Netzwerk-Nachricht-Klassen sollten von dieser Klasse erben. Diese Netzwerk-Unterklassen können dann über die bereitgestellte Methoden in NetworkConnection, NetworkClient und NetworkServer gesendet werden.	
MonoBehaviour	40
Dies ist die Basis Verhaltensklasse, die von jedem Skript standardmäßig implementiert wird.	
NetworkTransform	42
Diese Klasse wird benutzt, um die Bewegungen und Rotationen von Objekten über das Netzwerk zu synchronisieren.	
NetworkView	44
Die NetworkView Klasse definiert, auf welcher Weise bestimmte Objekte über das Netzwerk synchronisiert werden sollen.	
Object	45
Diese Klasse ist die Oberklasse zu allen Objekten in Unity.	
Pedestrian	47
Das Fußgänger Objekt welches ein Benutzer auswählen kann, um dieses in der Simulation zu verwenden.	
Quaternion	48
Quaternionen werden genutzt um Drehungen darzustellen.	
Rigidbody	49
Diese Klasse fügt eine physikalische Simulation zu der Position eines Objektes hinzu. Fügt man diese Komponente einem Objekt hinzu, so wird es unter die Kontrolle der physikalischen Engine von Unity gestellt.	
Scene	50
Laufzeitdatenstruktur für *.unity Dateien.	
Transform	51
Jedes Objekt einer Szene hat eine Transfer Komponente. Sie wird genutzt um die Position, Rotation und Größe des Objektes zu speichern.	
Vector3	52
Repräsentation von Vektoren im kartesischen Koordinatensystem.	

5.1 Klasse Behaviour



Dies ist die Oberklasse aller Behaviour Objekte. Behaviour Objekte sind Komponente welche an Objekte gehängt werden um dem Server mitzuteilen, wie dieses Objekt in der Simulation zu verwalten ist.

5.1.1 Deklaration

```
public class Behaviour
    erweitert UnityEngine.CoreModule.Component
```

5.1.2 Alle bekannten Unterklassen

VISSIMInterface , NetworkView (in [5.12](#), Seite [44](#)), NetworkTransform (in [5.11](#), Seite [42](#)), MonoBehaviour (in [5.10](#), Seite [40](#)), FirstPersonController (in [5.6](#), Seite [34](#)), CarUserControl (in [5.4](#), Seite [30](#)), Camera (in [5.2](#), Seite [28](#)), NetworkManager (in [6.6](#), Seite [64](#)), NetworkIdentity (in [6.5](#), Seite [62](#)), NetworkBehaviour (in [6.2](#), Seite [58](#)), CustomManager (in [6.1](#), Seite [55](#))

5.1.3 Attribute

- public bool **enabled**
 - Muss wahr sein wenn das Objekt aktualisiert werden soll.
- public bool **isActiveAndEnabled**
 - Ist wahr wenn das Verhalten aktiviert ist.

5.1.4 Konstruktoren

- Behaviour
 - public Behaviour()

5.1.5 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

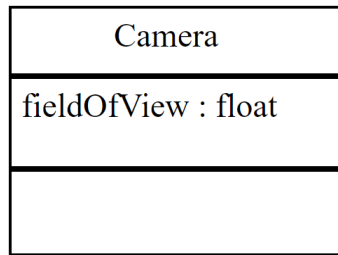
- public static void **Destroy()**
- public GameObject **gameObject**
- public void **GetComponent()**
- public string **tag**
- public Transform **transform**

5.1.6 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool **bool()**
- public static void **Destroy()**
- public void **GetInstanceID()**
- public HideFlags **hideFlags**
- public string **name**
- public string **ToString()**

5.2 Klasse Camera



Die Klasse Camera definiert die Sicht welche ein Benutzer in der Simulation hat.

5.2.1 Deklaration

```
public class Camera
    erweitert UnityEngine.CoreModule.Behaviour
```

5.2.2 Attribute

- public float **fieldOfView**
– Das Sichtfeld der Kamera angegeben in Grad.

5.2.3 Konstruktoren

- Camera
public Camera()

5.2.4 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in [5.1](#), Seite [26](#))

- public bool **enabled**
- public bool **isActiveAndEnabled**

5.2.5 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

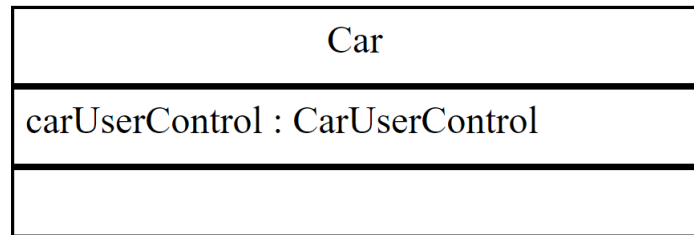
- public static void **Destroy()**
- public GameObject **gameObject**
- public void **GetComponent()**
- public string **tag**
- public Transform **transform**

5.2.6 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool **bool()**
- public static void **Destroy()**
- public void **GetInstanceID()**
- public HideFlags **hideFlags**
- public string **name**
- public string **ToString()**

5.3 Klasse Car



Das Autofahrer Objekt welches ein Benutzer auswählen kann um dieses in der Simulation zu verwenden.

5.3.1 Deklaration

```
public class Car
    erweitert UnityEngine.CoreModule.GameObject
```

5.3.2 Attribute

- `public CarUserControl carUserControl`
– Ein Skript zur Steuerung des Autos.

5.3.3 Konstruktoren

- `Car`
`public Car()`

5.3.4 Attribute geerbt von Klasse GameObject

`UnityEngine.CoreModule.GameObject` (in [5.7](#), Seite [36](#))

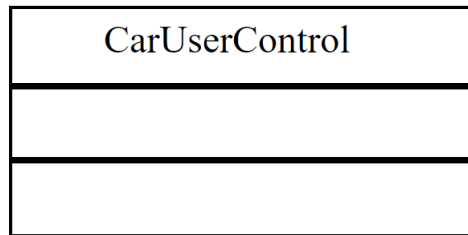
- `public bool activeInHierarchy`
- `public void AddComponent()`
- `public static void Find()`
- `public Rigidbody rigidBody`
- `public Scene scene`
- `public void SetActive()`
- `public string tag`
- `public Transform transform`

5.3.5 Attribute geerbt von Klasse Object

`UnityEngine.CoreModule.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

5.4 Klasse CarUserController



Diese Klasse ist für die Steuerung des Auto Objekt verantwortlich.

5.4.1 Deklaration

```
public class CarUserController
    erweitert UnityEngine.Networking.NetworkBehaviour
```

5.4.2 Konstruktoren

- **CarUserController**
public CarUserController()

5.4.3 Attribute geerbt von Klasse NetworkBehaviour

UnityEngine.Networking.NetworkBehaviour (in [6.2](#), Seite [58](#))

- public NetworkConnection connectionToClient
- public NetworkConnection connectionToServer
- public bool hasAuthority
- public int netId
- public void OnStartClient()
- public void OnStartServer()
- public int playerControllerId

5.4.4 Attribute geerbt von Klasse MonoBehaviour

UnityEngine.CoreModule.MonoBehaviour (in [5.10](#), Seite [40](#))

- public void OnDisable()
- public void OnEnable()
- public void OnFailedToConnect()
- public void OnGUI()
- public void Start()
- public void Update()

5.4.5 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in [5.1](#), Seite [26](#))

- public bool enabled
- public bool isActiveAndEnabled

5.4.6 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

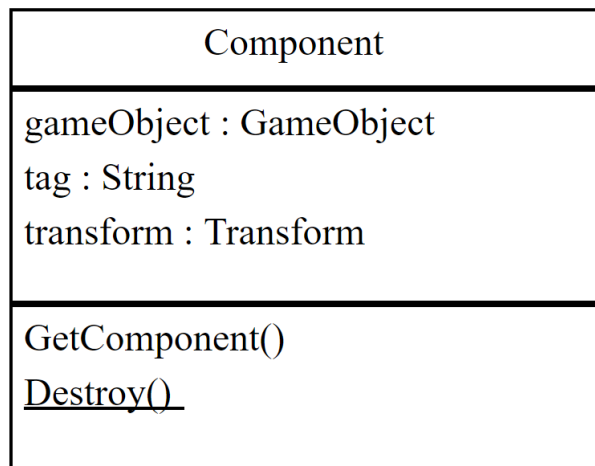
- `public static void Destroy()`
- `public GameObject gameObject`
- `public void GetComponent()`
- `public string tag`
- `public Transform transform`

5.4.7 Attribute geerbt von Klasse Object

`UnityEngine.CoreModule.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

5.5 Klasse Component



Die Oberklasse für alle Objektverknüpfungen.

5.5.1 Deklaration

```
public class Component
    erweitert UnityEngine.CoreModule.Object
```

5.5.2 Alle bekannten Unterklassen

VISSIMInterface , Transform (in 5.18, Seite 51), Rigidbody (in 5.16, Seite 49), NetworkView (in 5.12, Seite 44), NetworkTransform (in 5.11, Seite 42), MonoBehaviour (in 5.10, Seite 40), FirstPersonController (in 5.6, Seite 34), CarUserControl (in 5.4, Seite 30), Camera (in 5.2, Seite 28), Behaviour (in 5.1, Seite 26), NetworkManager (in 6.6, Seite 64), NetworkIdentity (in 6.5, Seite 62), NetworkBehaviour (in 6.2, Seite 58), CustomManager (in 6.1, Seite 55)

5.5.3 Attribute

- **public GameObject gameObject**
– Das Objekt von welchem diese Komponente ein Teil ist.
- **public string tag**
– Das Schlagwort der Komponente.
- **public Transform transform**
– Das sogenannte Transform Objekt. Es speichert die Positions- und die Rotationsdaten des Objektes.

5.5.4 Konstruktoren

- **Component**
public Component()

5.5.5 Methoden

- **Destroy**
public static void Destroy()

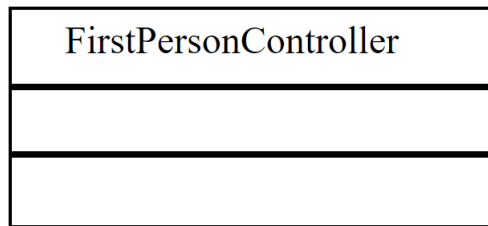
- **Beschreibung**
Diese Methode entfernt die Komponente.
- **GetComponent**
`public void GetComponent()`
 - **Beschreibung**
Diese Methode gibt die Komponente des Objektes zurück.

5.5.6 Attribute geerbt von Klasse Object

`UnityEngine.CoreModule.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

5.6 Klasse FirstPersonController



Diese Klasse steuert das Model des Benutzers aus einer FirstPerson Ansicht.

5.6.1 Deklaration

```
public class FirstPersonController
    erweitert UnityEngine.Networking.NetworkBehaviour
```

5.6.2 Konstruktoren

- **FirstPersonController**
public FirstPersonController()

5.6.3 Attribute geerbt von Klasse NetworkBehaviour

UnityEngine.Networking.NetworkBehaviour (in [6.2](#), Seite [58](#))

- public NetworkConnection connectionToClient
- public NetworkConnection connectionToServer
- public bool hasAuthority
- public int netId
- public void OnStartClient()
- public void OnStartServer()
- public int playerControllerId

5.6.4 Attribute geerbt von Klasse MonoBehaviour

UnityEngine.CoreModule.MonoBehaviour (in [5.10](#), Seite [40](#))

- public void OnDisable()
- public void OnEnable()
- public void OnFailedToConnect()
- public void OnGUI()
- public void Start()
- public void Update()

5.6.5 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in [5.1](#), Seite [26](#))

- public bool enabled
- public bool isActiveAndEnabled

5.6.6 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

- public static void **Destroy()**
- public GameObject **gameObject**
- public void **GetComponent()**
- public string **tag**
- public Transform **transform**

5.6.7 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool **bool()**
- public static void **Destroy()**
- public void **GetInstanceID()**
- public HideFlags **hideFlags**
- public string **name**
- public string **ToString()**

5.7 Klasse GameObject

GameObject
activeInHierarchy : Boolean scene : Scene tag : String transform : Transform rigidBody : Rigidbody
AddComponent() SetActive() <u>Find()</u>

Diese Klasse ist die Oberklasse aller Entitäten und Instanzen die in Unity Szenen genutzt werden.

5.7.1 Deklaration

```
public class GameObject
    erweitert UnityEngine.CoreModule.Object
```

5.7.2 Alle bekannten Unterklassen

Pedestrian (in [5.14](#), Seite [47](#)), Car (in [5.3](#), Seite [29](#))

5.7.3 Attribute

- `public bool activeInHierarchy`
– Ist wahr wenn das Objekt in der aktuellen Szene aktiv ist.
- `public Scene scene`
– Die Szene in der das Objekt aktiv ist.
- `public string tag`
– Das Schlagwort des Objektes.
- `public Transform transform`
– Das sogenannte Transform Objekt. Es speichert die Position und die Rotations Daten des Objektes.
- `public Rigidbody rigidBody`
– Das Kollisionsmodell und die physikalische Repräsentation des Game Objects.

5.7.4 Konstruktoren

- `GameObject`
`public GameObject()`

5.7.5 Methoden

- **AddComponent**

`public void AddComponent()`

- **Beschreibung**

Fügt dem Objekt eine neue Komponente hinzu.

- **Find**

`public static void Find()`

- **Beschreibung**

Findet ein Objekt über den angegebenen Namen und gibt dieses zurück.

- **SetActive**

`public void SetActive()`

- **Beschreibung**

Aktiviert oder deaktiviert ein Objekt. Dabei werden nur aktivierte Objekte in die Szene geladen.

5.7.6 Attribute geerbt von Klasse Object

`UnityEngine.CoreModule.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

5.8 Klasse HideFlags



Gibt an ob das Objekt versteckt, mit der Szene gespeichert werden oder vom Benutzer modifizierbar sein soll.

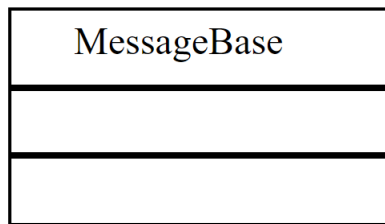
5.8.1 Deklaration

```
public class HideFlags
    erweitert UnityEngine.CoreModule.Object
```

5.8.2 Konstruktoren

- **HideFlags**
public HideFlags()

5.9 Klasse MessageBase



Netzwerk-Nachricht-Klassen sollten von dieser Klasse erben. Diese Netzwerk-Unterklassen können dann über die bereitgestellten Methoden in `NetworkConnection`, `NetworkClient` und `NetworkServer` gesendet werden.

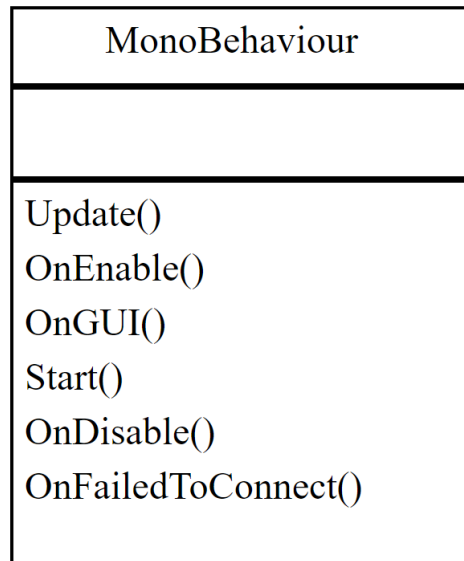
5.9.1 Deklaration

```
public class MessageBase
    erweitert UnityEngine.CoreModule.Object
```

5.9.2 Konstruktoren

- **MessageBase**
`public MessageBase()`

5.10 Klasse MonoBehaviour



Dies ist die Basis Verhaltensklasse welche von jedem Skript standardmäßig implementiert wird.

5.10.1 Deklaration

```
public class MonoBehaviour
    erweitert UnityEngine.CoreModule.Behaviour
```

5.10.2 Alle bekannten Unterklassen

VISSIMInterface , NetworkTransform (in 5.11, Seite 42), FirstPersonController (in 5.6, Seite 34), CarUserControl (in 5.4, Seite 30), NetworkManager (in 6.6, Seite 64), NetworkIdentity (in 6.5, Seite 62), NetworkBehaviour (in 6.2, Seite 58), CustomManager (in 6.1, Seite 55)

5.10.3 Konstruktoren

- **MonoBehaviour**
public MonoBehaviour()

5.10.4 Methoden

- **OnDisable**
public void OnDisable()
– **Beschreibung**
Diese Methode wird ausgeführt, wenn das Objekt deaktiviert wird.
- **OnEnable**
public void OnEnable()
– **Beschreibung**
Diese Methode wird ausgeführt, sobald das Skript aktiviert wird.
- **OnFailedToConnect**
public void OnFailedToConnect()
– **Beschreibung**

Diese Methode wird ausgeführt, wenn ein Fehler in der Verbindung erkannt wird.

- **OnGUI**

```
public void OnGUI()
```

- **Beschreibung**

Diese Methode verwaltet Events der Benutzeroberfläche.

- **Start**

```
public void Start()
```

- **Beschreibung**

Wird ausgeführt wenn das Skript aktiviert wird. Dies geschieht vor irgendeiner anderen Aktualisierungsmethode.

- **Update**

```
public void Update()
```

- **Beschreibung**

Wird nach jedem Bild aufgerufen, um unterbrechungsfreie Aktualisierungen zu erlauben.

5.10.5 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in [5.1](#), Seite [26](#))

- public bool enabled
- public bool isActiveAndEnabled

5.10.6 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

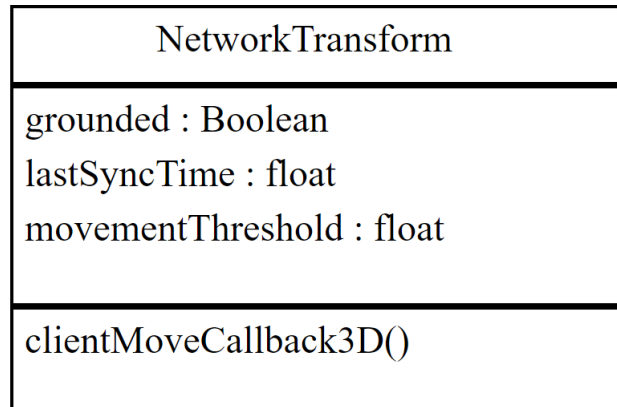
- public static void Destroy()
- public GameObject gameObject
- public void GetComponent()
- public string tag
- public Transform transform

5.10.7 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool bool()
- public static void Destroy()
- public void GetInstanceID()
- public HideFlags hideFlags
- public string name
- public string ToString()

5.11 Klasse NetworkTransform



Diese Klasse wird benutzt, um die Bewegungen und Rotationen von Objekten über das Netzwerk zu synchronisieren.

5.11.1 Deklaration

```
public class NetworkTransform
    erweitert UnityEngine.Networking.NetworkBehaviour
```

5.11.2 Alle bekannten Unterklassen

VISSIMInterface

5.11.3 Attribute

- **public bool grounded**
 - Ist wahr, falls das Objekt sich auf einer Oberfläche befindet.
- **public float lastSyncTime**
 - Der letzte Zeitpunkt zu dem ein Paket, zum Zwecke der Bewegung des Objektes, angekommen ist.
- **public float movementTheshold**
 - Die Distanz, die sich ein Objekt bewegen kann, ohne dass eine Synchronisation mit dem Server erfolgt.

5.11.4 Konstruktoren

- **NetworkTransform**

```
public NetworkTransform()
```

5.11.5 Methoden

- **clientMoveCallback3D**

```
public void clientMoveCallback3D()
```

 - **Beschreibung**
Wird verwendet, um Bewegungen über den Server zu verifizieren.

5.11.6 Attribute geerbt von Klasse NetworkBehaviour

UnityEngine.Networking.NetworkBehaviour (in [6.2](#), Seite [58](#))

- public NetworkConnection connectionToClient
- public NetworkConnection connectionToServer
- public bool hasAuthority
- public int netId
- public void OnStartClient()
- public void OnStartServer()
- public int playerControllerId

5.11.7 Attribute geerbt von Klasse MonoBehaviour

UnityEngine.CoreModule.MonoBehaviour (in [5.10](#), Seite [40](#))

- public void OnDisable()
- public void OnEnable()
- public void OnFailedToConnect()
- public void OnGUI()
- public void Start()
- public void Update()

5.11.8 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in [5.1](#), Seite [26](#))

- public bool enabled
- public bool isActiveAndEnabled

5.11.9 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

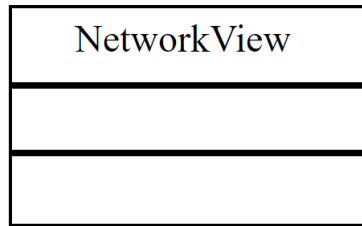
- public static void Destroy()
- public GameObject gameObject
- public void GetComponent()
- public string tag
- public Transform transform

5.11.10 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool bool()
- public static void Destroy()
- public void GetInstanceID()
- public HideFlags hideFlags
- public string name
- public string ToString()

5.12 Klasse NetworkView



Die NetworkView Klasse definiert, auf welcher Weise bestimmte Objekte über das Netzwerk synchronisiert werden sollen.

5.12.1 Deklaration

```
public class NetworkView
    erweitert UnityEngine.CoreModule.Behaviour
```

5.12.2 Konstruktoren

- **NetworkView**
public NetworkView()

5.12.3 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in [5.1](#), Seite [26](#))

- public bool enabled
- public bool isActiveAndEnabled

5.12.4 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

- public static void Destroy()
- public GameObject gameObject
- public void GetComponent()
- public string tag
- public Transform transform

5.12.5 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool bool()
- public static void Destroy()
- public void GetInstanceID()
- public HideFlags hideFlags
- public string name
- public string ToString()

5.13 Klasse Object

Object
hideFlags : HideFlags name : String
GetInstanceID() ToString() : String <u>Destroy()</u> bool() : Boolean

Diese Klasse ist die Oberklasse zu allen Objekten in Unity.

5.13.1 Deklaration

```
public class Object
    erweitert UnityEngine.CoreModule.Object
```

5.13.2 Alle bekannten Unterklassen

VISSIMInterface , Transform (in 5.18, Seite 51), Rigidbody (in 5.16, Seite 49), Pedestrian (in 5.14, Seite 47), NetworkView (in 5.12, Seite 44), NetworkTransform (in 5.11, Seite 42), MonoBehaviour (in 5.10, Seite 40), GameObject (in 5.7, Seite 36), FirstPersonController (in 5.6, Seite 34), Component (in 5.5, Seite 32), CarUserControl (in 5.4, Seite 30), Car (in 5.3, Seite 29), Camera (in 5.2, Seite 28), Behaviour (in 5.1, Seite 26), NetworkManager (in 6.6, Seite 64), NetworkIdentity (in 6.5, Seite 62), NetworkBehaviour (in 6.2, Seite 58), CustomManager (in 6.1, Seite 55)

5.13.3 Attribute

- `public HideFlags hideFlags`
 - Eine Flagge macht es möglich, zu identifizieren ob ein Objekt versteckt, gespeichert oder vom Benutzer modifizierbar sein soll.
- `public string name`
 - Der Name des Objekts.

5.13.4 Konstruktoren

- `Object`
 - `public Object()`

5.13.5 Methoden

- `bool`
 - `public bool bool()`
 - Beschreibung

Diese Methode gibt wahr zurück, wenn das Objekt existiert.

- **Destroy**

`public static void Destroy()`

- **Beschreibung**

Diese Funktion entfernt ein Objekt, Komponente oder ein Asset.

- **GetInstanceID**

`public void GetInstanceID()`

- **Beschreibung**

Diese Methode gibt die ID des Objektes zurück. Diese ID ist garantiert einzigartig.

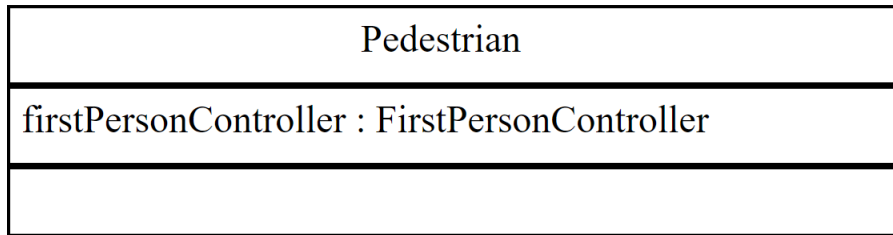
- **ToString**

`public string ToString()`

- **Beschreibung**

Diese Methode gibt den Namen des Objektes zurück.

5.14 Klasse Pedestrian



Das Fußgänger Objekt welches ein Benutzer auswählen kann um dieses in der Simulation zu verwenden.

5.14.1 Deklaration

```
public class Pedestrian
    erweitert UnityEngine.GameObject
```

5.14.2 Attribute

- `public FirstPersonController firstPersonController`
– Die Kontroll-Klasse, die den Fußgänger steuern soll.

5.14.3 Konstruktoren

- `Pedestrian`
`public Pedestrian()`

5.14.4 Attribute geerbt von Klasse GameObject

`UnityEngine.GameObject` (in [5.7](#), Seite [36](#))

- `public bool activeInHierarchy`
- `public void AddComponent()`
- `public static void Find()`
- `public Rigidbody rigidBody`
- `public Scene scene`
- `public void SetActive()`
- `public string tag`
- `public Transform transform`

5.14.5 Attribute geerbt von Klasse Object

`UnityEngine.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

5.15 Klasse Quaternion

Quaternion
eulerAngles : Vector3 w : float x : float y : float z : float
SetFromRotation(inRotation : Vector3,outRotation : Vector3)

Quaternionen werden genutzt um Drehungen darzustellen.

5.15.1 Deklaration

```
public class Quaternion
    erweitert UnityEngine.CoreModule.Object
```

5.15.2 Attribute

- **public Vector3 eulerAngles**
– Gibt die Euler Repräsentation der Rotation wieder.
- **public float w**
– W Komponente der Quaternion.
- **public float x**
– X Komponente der Quaternion. Sollte nicht direkt modifiziert werden.
- **public float y**
– Y Komponente der Quaternion. Sollte nicht direkt modifiziert werden.
- **public float z**
– Z Komponente der Quaternion. Sollte nicht direkt modifiziert werden.

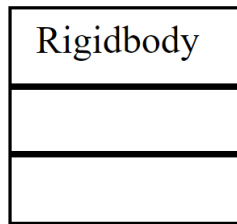
5.15.3 Konstruktoren

- **Quaternion**
public Quaternion()

5.15.4 Methoden

- **SetFromRotation**
public void SetFromRotation(Vector3 inRotation,Vector3 outRotation)
– **Beschreibung**
Erzeugt eine Rotation, welche den Vektor inRotation in den Vektor outRotation überführt.

5.16 Klasse Rigidbody



Diese Klasse fügt eine physikalische Simulation zu der Position eines Objektes hinzu. Fügt man diese Komponente einem Objekt hinzu, so wird es unter Kontrolle der physikalischen Engine von Unity gestellt.

5.16.1 Deklaration

```
public class Rigidbody
    erweitert UnityEngine.CoreModule.Component
```

5.16.2 Attribute

- `public Car myCar`

5.16.3 Konstruktoren

- **Rigidbody**
`public Rigidbody()`

5.16.4 Attribute geerbt von Klasse Component

`UnityEngine.CoreModule.Component` (in [5.5](#), Seite [32](#))

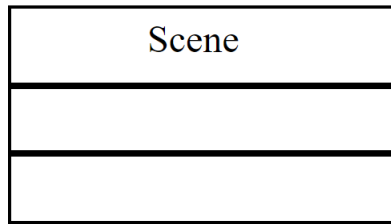
- `public static void Destroy()`
- `public GameObject gameObject`
- `public void GetComponent()`
- `public string tag`
- `public Transform transform`

5.16.5 Attribute geerbt von Klasse Object

`UnityEngine.CoreModule.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

5.17 Klasse Scene



Laufzeitdatenstruktur für *.unity Dateien.

5.17.1 Deklaration

```
public class Scene
    erweitert UnityEngine.CoreModule.Object
```

5.17.2 Konstruktoren

- **Scene**
public Scene()

5.18 Klasse Transform

Transform
position : Vector3 rotation : Quaternion

Jedes Objekt einer Szene hat eine Transfer Komponente. Sie wird genutzt um die Position, Rotation und Größe des Objektes zu speichern.

5.18.1 Deklaration

```
public class Transform
    erweitert UnityEngine.Component
```

5.18.2 Attribute

- **public Vector3 position**
– Die Position des Objektes im Format (x,y,z).
- **public Quaternion rotation**
– Die Rotation des Objektes.

5.18.3 Konstruktoren

- **Transform**
`public Transform()`

5.18.4 Attribute geerbt von Klasse Component

`UnityEngine.Component` (in [5.5](#), Seite [32](#))

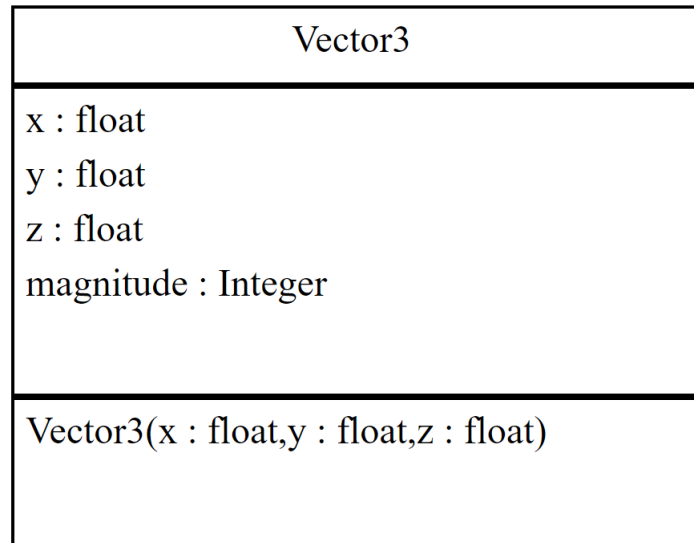
- **public static void Destroy()**
- **public GameObject gameObject**
- **public void GetComponent()**
- **public string tag**
- **public Transform transform**

5.18.5 Attribute geerbt von Klasse Object

`UnityEngine.Object` (in [5.13](#), Seite [45](#))

- **public bool bool()**
- **public static void Destroy()**
- **public void GetInstanceID()**
- **public HideFlags hideFlags**
- **public string name**
- **public string ToString()**

5.19 Klasse Vector3



Repräsentation von Vektoren im kartesischen Koordinatensystem.

5.19.1 Deklaration

```
public class Vector3
    erweitert UnityEngine.CoreModule.Object
```

5.19.2 Attribute

- **public float x**
– Die x Komponente des Vektors.
- **public float y**
– Die y Komponente des Vektors.
- **public float z**
– Die z Komponente des Vektors.
- **public int magnitude**
– Gibt den Betrag des Vektors wieder.

5.19.3 Konstruktoren

- **Vector3**
public Vector3()

5.19.4 Methoden

- **Vector3**
public void Vector3(float x,float y,float z)
– **Beschreibung**
Erstellt einen neuen Vektor mit gegebenen Koordinaten.

6 Paket UnityEngine.Networking

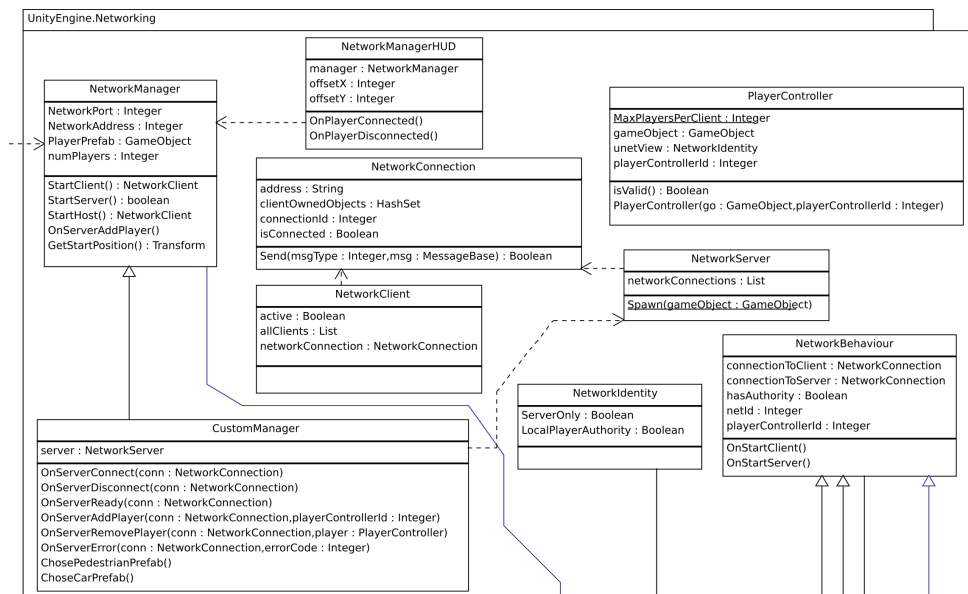


Abbildung 7: Klassendiagramm - Paket UnityEngine.Networking

Beschreibung

Das UnityEngine.Networking Paket beinhaltet alle wichtigen Klassen die zur Umsetzung der Netzwerkfähigkeit der Simulation benötigt werden. Es beinhaltet die wichtigste Komponente, den NetworkManager, und beinhaltet alle mit diesem in Relation stehenden Klassen. Diese Klassen sind bereits von Unity implementiert und werden von uns an die Anforderungen des Projekts angepasst.

Paket Inhalt

Seite

Klassen

CustomManager 55

Der CustomManager ist eine spezielle Form des NetworkManager. Dieser wird speziell für das Projekt angepasst und hat mehr Funktionen als der üblicherweise eingesetzte NetworkManager.

NetworkBehaviour 58

Die NetworkBehaviour Klasse wird von anderen Klassen geerbt welche Netzwerk Fähigkeit beeinflussen. Jedes Objekt braucht ein NetworkBehaviour und eine NetworkIdentity damit die Netzwerkfunktionalität gewährleistet ist.

NetworkClient 60

Die Transport Level Netzwerk Klasse. Sie prüft ob die IP Adresse korrekt ist und verbindet mithilfe des Internet Protokolls.

NetworkConnection 61

Eine Netzwerk Verbindung ist eine Verbindung zwischen einem Server und einem Client.

NetworkIdentity 62

Die NetzwerkIdentity wird benutzt um Objekte über die Simulation hinweg zu identifizieren. Diese hilft dem Server Informationen über das Netzwerk zu synchronisieren.

NetworkManager 64

Der NetworkManager ist die Hauptkomponente der Netzwerkverbindung mit anderen Benutzern. Der NetworkManager ist verantwortlich für das Management aller Netzwerk bezogenen Klassen.

NetworkManagerHUD 67

Das NetworkManagerHUD ist ein einfaches Benutzerinterface welches während der Ausführung der Simulation angezeigt wird. Dieses wird hauptsächlich während der Implementierungsphase verwendet und später durch die Graphische Benutzer Oberfläche ersetzt.

NetworkServer 68

Der Netzwerk Server verwaltet Verbindungen zwischen weit entfernten Clients. Es gibt auch eine lokale Verbindung für lokal verbundene Clients.

PlayerController 69

Der PlayerController wird benutzt um das Benutzer Model in der First-Person und Third-Person Ansicht zu kontrollieren.

6.1 Klasse CustomManager

CustomManager
server : NetworkServer
OnServerConnect(conn : NetworkConnection) OnServerDisconnect(conn : NetworkConnection) OnServerReady(conn : NetworkConnection) OnServerAddPlayer(conn : NetworkConnection,playerControllerId : Integer) OnServerRemovePlayer(conn : NetworkConnection,player : PlayerController) OnServerError(conn : NetworkConnection,errorCode : Integer) ChosePedestrianPrefab() ChoseCarPrefab()

Der CustomManager ist eine spezielle Form des NetworkManager. Dieser wird speziell für das Projekt angepasst und hat mehr Funktionen als der üblicherweise eingesetzte NetworkManager.

6.1.1 Deklaration

```
public class CustomManager
    erweitert UnityEngine.Networking.NetworkManager
```

6.1.2 Konstruktoren

- **CustomManager**
public CustomManager()

6.1.3 Methoden

- **ChoseCarPrefab**
public void ChoseCarPrefab()
– **Beschreibung**
Wird aufgerufen, wenn der Benutzer das Fahrer-Modell gewählt hat.
- **ChosePedestrianPrefab**
public void ChosePedestrianPrefab()
– **Beschreibung**
Wird aufgerufen, wenn der Benutzer das Fußgänger-Modell ausgewählt hat.
- **OnServerAddPlayer**
public void OnServerAddPlayer(NetworkConnection conn,int playerControllerId)
– **Beschreibung**
Wird aufgerufen, wenn der Server einen neuen Benutzer hinzufügt. Für den neuverbundenen Client wird ein Modell in die Szene eingefügt.
- **OnServerConnect**
public void OnServerConnect(NetworkConnection conn)
– **Beschreibung**
Diese Methode wird aufgerufen, sobald sich ein neuer Client mit dem Server verbindet.

- **OnServerDisconnect**

```
public void OnServerDisconnect(NetworkConnection conn)
```

- **Beschreibung**

Diese Methode wird aufgerufen, sobald ein Client die Verbindung mit dem Server trennt.

- **OnServerError**

```
public void OnServerError(NetworkConnection conn,int errorCode)
```

- **Beschreibung**

Diese Methode wird aufgerufen, falls ein Netzwerkfehler erkannt wird.

- **OnServerReady**

```
public void OnServerReady(NetworkConnection conn)
```

- **Beschreibung**

Wird ausgeführt, wenn ein Client bereit ist dem Server beizutreten.

- **OnServerRemovePlayer**

```
public void OnServerRemovePlayer(NetworkConnection conn,PlayerController player)
```

- **Beschreibung**

Diese Methode wird aufgerufen, sobald ein Client einen Benutzer aus der Simulation entfernt.

6.1.4 Attribute geerbt von Klasse NetworkManager

UnityEngine.Networking.NetworkManager (in [6.6](#), Seite [64](#))

- public Transform **GetStartPosition()**
- public int **NetworkAddress**
- public int **NetworkPort**
- public int **numPlayers**
- public void **OnServerAddPlayer()**
- public GameObject **PlayerPrefab**
- public NetworkClient **StartClient()**
- public NetworkClient **StartHost()**
- public bool **StartServer()**

6.1.5 Attribute geerbt von Klasse MonoBehaviour

UnityEngine.CoreModule.MonoBehaviour (in [5.10](#), Seite [40](#))

- public void **OnDisable()**
- public void **OnEnable()**
- public void **OnFailedToConnect()**
- public void **OnGUI()**
- public void **Start()**
- public void **Update()**

6.1.6 Attribute geerbt von Klasse Behaviour

UnityEngine.CoreModule.Behaviour (in [5.1](#), Seite [26](#))

- public bool **enabled**
- public bool **isActiveAndEnabled**

6.1.7 Attribute geerbt von Klasse Component

UnityEngine.CoreModule.Component (in [5.5](#), Seite [32](#))

- public static void **Destroy()**
- public GameObject **gameObject**
- public void **GetComponent()**
- public string **tag**
- public Transform **transform**

6.1.8 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool **bool()**
- public static void **Destroy()**
- public void **GetInstanceID()**
- public HideFlags **hideFlags**
- public string **name**
- public string **ToString()**

6.2 Klasse NetworkBehaviour

NetworkBehaviour
connectionToClient : NetworkConnection connectionToServer : NetworkConnection hasAuthority : Boolean netId : Integer playerControllerId : Integer
OnStartClient() OnStartServer()

Die NetworkBehaviour Klasse wird von allen Klassen geerbt, die Netzwerk-Fähigkeit beeinflussen. Jedes Objekt braucht ein NetworkBehaviour und eine NetworkIdentity damit die Netzwerkfunktionalität gewährleistet ist.

6.2.1 Deklaration

```
public class NetworkBehaviour
    erweitert UnityEngine.CoreModule.MonoBehaviour
```

6.2.2 Alle bekannten Unterklassen

VISSIMInterface , NetworkTransform (in 5.11, Seite 42), FirstPersonController (in 5.6, Seite 34), CarUserControl (in 5.4, Seite 30)

6.2.3 Attribute

- **public NetworkConnection connectionToClient**
 - Behinhaltet Informationen wie Beispielsweise die NetworkIdentity, IP Adresse oder den Bereitschaftsstatus.
- **public NetworkConnection connectionToServer**
 - Beinhaltet Informationen über den Server wie Beispielsweise den Port oder die IP Adresse.
- **public bool hasAuthority**
 - Ist wahr, falls das Objekt ein autoritäres Objekt im aktuellen Netzwerk ist.
- **public int netId**
 - Die spezielle Netzwerk Identifikationsnummer.
- **public int playerControllerId**
 - Die spezielle Identifikationsnummer des Benutzers, welche zu diesem Objekt gehört.

6.2.4 Konstruktoren

- **NetworkBehaviour**

```
public NetworkBehaviour()
```

6.2.5 Methoden

- **OnStartClient**
`public void OnStartClient()`
 - **Beschreibung**
Diese Methode wird ausgeführt, wenn Objekte vom Client aktiviert worden sind.
- **OnStartServer**
`public void OnStartServer()`
 - **Beschreibung**
Diese Methode wird ausgeführt, nachdem Objekte auf dem Server aktiv werden.

6.2.6 Attribute geerbt von Klasse MonoBehaviour

`UnityEngine.CoreModule.MonoBehaviour` (in [5.10](#), Seite [40](#))

- `public void OnDisable()`
- `public void OnEnable()`
- `public void OnFailedToConnect()`
- `public void OnGUI()`
- `public void Start()`
- `public void Update()`

6.2.7 Attribute geerbt von Klasse Behaviour

`UnityEngine.CoreModule.Behaviour` (in [5.1](#), Seite [26](#))

- `public bool enabled`
- `public bool isActiveAndEnabled`

6.2.8 Attribute geerbt von Klasse Component

`UnityEngine.CoreModule.Component` (in [5.5](#), Seite [32](#))

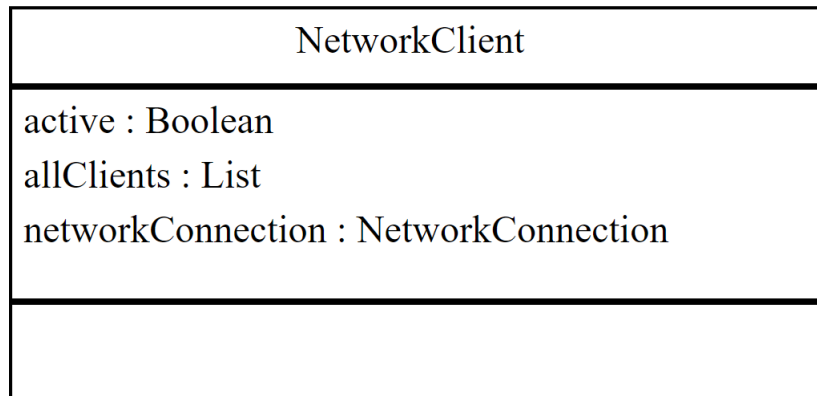
- `public static void Destroy()`
- `public GameObject gameObject`
- `public void GetComponent()`
- `public string tag`
- `public Transform transform`

6.2.9 Attribute geerbt von Klasse Object

`UnityEngine.CoreModule.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

6.3 Klasse NetworkClient



Die Transport Level Netzwerk Klasse. Sie prüft ob die IP Adresse korrekt ist und verbindet mithilfe des Internet Protokolls.

6.3.1 Deklaration

```
public class NetworkClient
    erweitert UnityEngine.CoreModule.Object
```

6.3.2 Attribute

- **public bool active**
 - Ist wahr, sollte der Client gerade aktiv sein.
- **public System.Collections.Generic.List<NetworkClient> allClients**
 - Eine Liste an aktiven Netzwerk Clients, in der aktuellen Ausführung.
- **public NetworkConnection networkConnection**
 - Die Netzwerkverbindung des Clients zu einem Server.

6.3.3 Konstruktoren

- **NetworkClient**

```
public NetworkClient()
```

6.4 Klasse NetworkConnection

NetworkConnection
address : String clientOwnedObjects : HashSet connectionId : Integer isConnected : Boolean
Send(msgType : Integer,msg : MessageBase) : Boolean

Eine Netzwerk Verbindung ist eine Verbindung zwischen einem Server und einem Client.

6.4.1 Deklaration

```
public class NetworkConnection
    erweitert UnityEngine.CoreModule.Object
```

6.4.2 Attribute

- **public string address**
– Die IP Adresse der Netzwerk Verbindung.
- **public System.Collections.Generic.HashSet<NetworkInstanceId> clientOwnedObjects**
– Eine Liste an networkidentity Objekten die zu dieser Verbindung gehören.
- **public int connectionId**
– Die spezielle Identifikationsnummer dieser Verbindung.
- **public bool isConnected**
– Ist wahr, sollte die Verbindung zu einem Endpunkt bestehen.

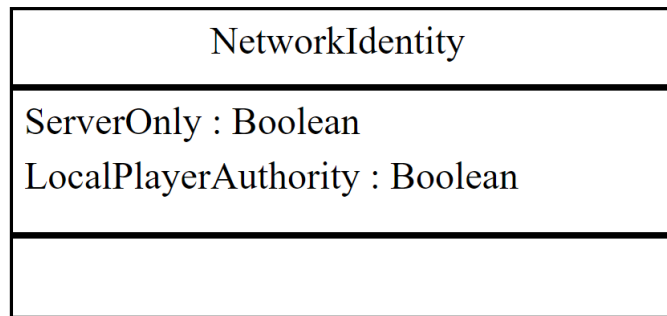
6.4.3 Konstruktoren

- **NetworkConnection**
public NetworkConnection()

6.4.4 Methoden

- **Send**
public bool Send(int msgType,UnityEngine.CoreModule.MessageBase msg)
– **Beschreibung**
Diese Methode sendet eine Netzwerk Nachricht, mit einer speziellen Nachrichtenidentifikationsnummer.

6.5 Klasse NetworkIdentity



Die NetworkIdentity wird benutzt um Objekte über die Simulation hinweg zu identifizieren. Diese hilft dem Server Informationen über das Netzwerk zu synchronisieren.

6.5.1 Deklaration

```
public class NetworkIdentity
    erweitert UnityEngine.CoreModule.MonoBehaviour
```

6.5.2 Attribute

- **public bool ServerOnly**
– Ist wahr, wenn das Objekt nur auf dem Server zu sehen sein soll.
- **public bool LocalPlayerAuthority**
– Ist wahr, wenn das Objekt von einem Client gesteuert wird.

6.5.3 Konstruktoren

- **NetworkIdentity**
`public NetworkIdentity()`

6.5.4 Attribute geerbt von Klasse MonoBehaviour

`UnityEngine.CoreModule.MonoBehaviour` (in [5.10](#), Seite [40](#))

- **public void OnDisable()**
- **public void OnEnable()**
- **public void OnFailedToConnect()**
- **public void OnGUI()**
- **public void Start()**
- **public void Update()**

6.5.5 Attribute geerbt von Klasse Behaviour

`UnityEngine.CoreModule.Behaviour` (in [5.1](#), Seite [26](#))

- **public bool enabled**
- **public bool isActiveAndEnabled**

6.5.6 Attribute geerbt von Klasse Component

`UnityEngine.CoreModule.Component` (in [5.5](#), Seite [32](#))

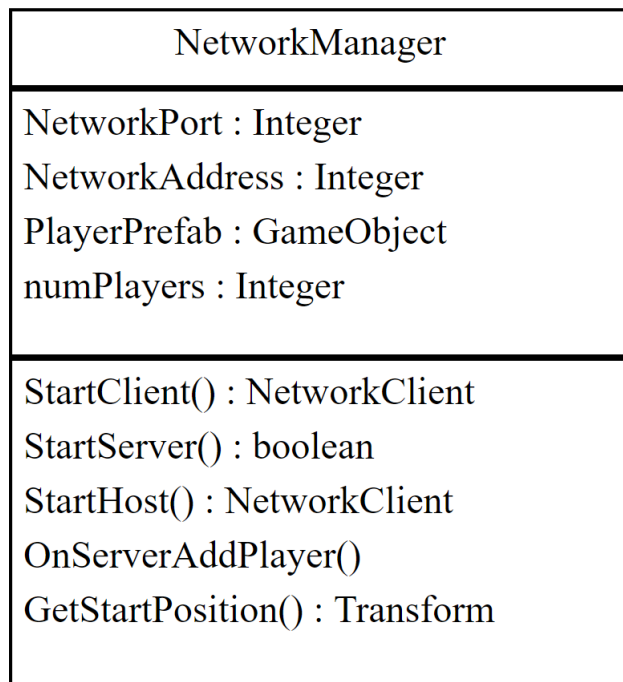
- public static void **Destroy()**
- public GameObject **gameObject**
- public void **GetComponent()**
- public string **tag**
- public Transform **transform**

6.5.7 Attribute geerbt von Klasse Object

UnityEngine.CoreModule.Object (in [5.13](#), Seite [45](#))

- public bool **bool()**
- public static void **Destroy()**
- public void **GetInstanceID()**
- public HideFlags **hideFlags**
- public string **name**
- public string **ToString()**

6.6 Klasse NetworkManager



Der NetworkManager ist die Hauptkomponente der Netzwerkverbindung mit anderen Benutzern. Der NetworkManager ist verantwortlich für das Management aller Netzwerk bezogenen Klassen.

6.6.1 Deklaration

```
public class NetworkManager
    erweitert UnityEngine.CoreModule.MonoBehaviour
```

6.6.2 Alle bekannten Unterklassen

CustomManager (in [6.1](#), Seite [55](#))

6.6.3 Attribute

- **public int NetworkPort**
– Definiert den Netzwerk-Port des Hosts.
- **public int NetworkAddress**
– Definiert die IP Adresse des Hosts.
- **public UnityEngine.CoreModule.GameObject PlayerPrefab**
– Das GameObject, welches für das Modell des Benutzers benutzt wird.
- **public int numPlayers**
– Gibt die Anzahl der Clients an, welche sich momentan auf dem Server befinden.

6.6.4 Konstruktoren

- **NetworkManager**
`public NetworkManager()`

6.6.5 Methoden

- **GetStartPosition**
`public UnityEngine.CoreModule.Transform GetStartPosition()`
 - **Beschreibung**
 Diese Methode findet eine Startposition für das Benutzer Modell.
- **OnServerAddPlayer**
`public void OnServerAddPlayer()`
 - **Beschreibung**
 Diese Methode wird ausgeführt wenn ein mit dem Server verbundener Client einen Benutzer hinzufügen möchte. Diese Methode erstellt für den Benutzer ein neues Modell in der Szene.
- **StartClient**
`public NetworkClient StartClient()`
 - **Beschreibung**
 Diese Methode startet einen neuen Netzwerk Client, welcher sich mit einer Netzwerkadresse und einem Netzwerk Port zu einem Host verbindet.
- **StartHost**
`public NetworkClient StartHost()`
 - **Beschreibung**
 Diese Methode startet einen neuen Server, sowie einen Client in der gleichen Programmausführung.
- **StartServer**
`public bool StartServer()`
 - **Beschreibung**
 Diese Methode startet eine neue Server Instanz und gibt wahr zurück, falls der Server gestartet ist.

6.6.6 Attribute geerbt von Klasse MonoBehaviour

`UnityEngine.CoreModule.MonoBehaviour` (in [5.10](#), Seite [40](#))

- `public void OnDisable()`
- `public void OnEnable()`
- `public void OnFailedToConnect()`
- `public void OnGUI()`
- `public void Start()`
- `public void Update()`

6.6.7 Attribute geerbt von Klasse Behaviour

`UnityEngine.CoreModule.Behaviour` (in [5.1](#), Seite [26](#))

- `public bool enabled`
- `public bool isActiveAndEnabled`

6.6.8 Attribute geerbt von Klasse Component

`UnityEngine.CoreModule.Component` (in [5.5](#), Seite [32](#))

- `public static void Destroy()`
- `public GameObject gameObject`
- `public void GetComponent()`
- `public string tag`
- `public Transform transform`

6.6.9 Attribute geerbt von Klasse Object

`UnityEngine.CoreModule.Object` (in [5.13](#), Seite [45](#))

- `public bool bool()`
- `public static void Destroy()`
- `public void GetInstanceID()`
- `public HideFlags hideFlags`
- `public string name`
- `public string ToString()`

6.7 Klasse NetworkManagerHUD

NetworkManagerHUD
manager : NetworkManager offsetX : Integer offsetY : Integer
OnPlayerConnected() OnPlayerDisconnected()

Das NetworkManagerHUD ist ein einfaches Benutzerinterface welches während der Ausführung der Simulation angezeigt wird. Dieses wird hauptsächlich während der Implementierungsphase verwendet und später durch die graphische Benutzer Oberfläche ersetzt.

6.7.1 Deklaration

```
public class NetworkManagerHUD
    erweitert UnityEngine.CoreModule.Object
```

6.7.2 Attribute

- **public NetworkManager manager**
– Der NetworkManager, welcher momentan benutzt wird.
- **public int offsetX**
– Der X-Achsen Abstand des NetworkManagerHUD.
- **public int offsetY**
– Der Y-Achsen Abstand des NetworkManagerHUD.

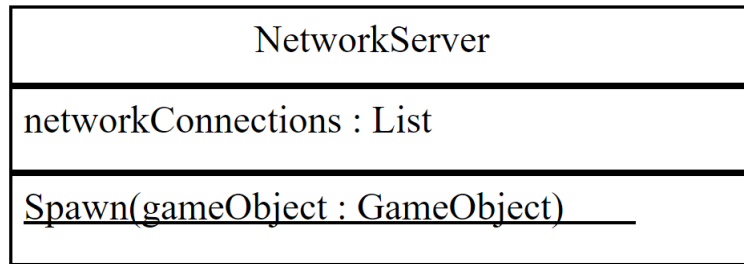
6.7.3 Konstruktoren

- **NetworkManagerHUD**
public NetworkManagerHUD()

6.7.4 Methoden

- **OnPlayerConnected**
public void OnPlayerConnected()
– **Beschreibung**
Diese Methode wird aufgerufen, wenn dem Server ein neuer Benutzer hinzugefügt wird.
- **OnPlayerDisconnected**
public void OnPlayerDisconnected()
– **Beschreibung**
Diese Methode wird aufgerufen, sobald ein Benutzer vom Server entfernt wird.

6.8 Klasse NetworkServer



Der Netzwerk Server verwaltet Verbindungen zwischen weit entfernten Clients. Es gibt auch eine lokale Verbindung für lokal verbundene Clients.

6.8.1 Deklaration

```
public class NetworkServer
    erweitert UnityEngine.CoreModule.Object
```

6.8.2 Attribute

- `public System.Collections.Generic.List<NetworkConnection> networkConnections`
– Eine Liste von Netzwerkverbindungen, die dieser Server gerade verwaltet.

6.8.3 Konstruktoren

- `NetworkServer`
`public NetworkServer()`

6.8.4 Methoden

- `Spawn`
`public static void Spawn(UnityEngine.CoreModule.GameObject gameObject)`
– **Beschreibung**
Verteilt das übergebene GameObject an alle verbundenen Clients.

6.9 Klasse PlayerController

PlayerController
<u>MaxPlayersPerClient : Integer</u> gameObject : GameObject unetView : NetworkIdentity playerControllerId : Integer
isValid() : Boolean PlayerController(go : GameObject,playerControllerId : Integer)

Der PlayerController wird benutzt um das Benutzer Model in der First-Person und in der Third-Person Ansicht zu kontrollieren.

6.9.1 Deklaration

```
public class PlayerController
    erweitert UnityEngine.CoreModule.Object
```

6.9.2 Attribute

- **public static int MaxPlayersPerClient**
– Die Maximale Anzahl an Benutzer pro Client.
- **public UnityEngine.CoreModule.GameObject gameObject**
– Die GameObject Klasse die den Spieler repräsentieren soll.
- **public NetworkIdentity unetView**
– Die Netzwerk Identität die zum GameObject gameObject gehört.
- **public int playerId**
– Identifikationsnummer um verschiedene Benutzer zu differenzieren.

6.9.3 Konstruktoren

- **PlayerController**
public PlayerController()
- **PlayerController**
public PlayerController(UnityEngine.CoreModule.GameObject go,int playerId)
– **Beschreibung**
Initialisiert das GameObject und die Identifikationsnummer.

6.9.4 Methoden

- **isValid**
public bool isValid()
– **Beschreibung**
Gibt wahr zurück wenn die ControllerId ungleich -1, d.h. initialisiert ist.

7 Abläufe

7.1 Sequenzdiagramme

7.1.1 Host startet und beendet die Simulation

Dieses Sequenzdiagramm beschreibt ein Szenario aus dem Pflichtenheft. Das Szenario besteht daraus, dass der Host das Programm startet und einen Host-Server erstellt. Danach beendet er Unity und damit auch den Host-Server.

Nach dem Start des Programms, erzeugt Unity die für die Ausführung benötigten Komponenten. In diesem Diagramm wird lediglich eine vereinfachte Version des tatsächlichen Ablaufes dargestellt. Unity wird hier als eine Klasse modelliert. Dies entspricht leider nicht der Realität, musste aber den Umständen entsprechend so modelliert werden. Der Grund für dieses stark vereinfachte Modell ist, dass Unity nur die Klassen dokumentiert, die auch später von dem Anwendungsentwickler verändert werden. Die Engine selbst bleibt dabei verborgen. Es ist also nicht nachzuvollziehen, wie der Programmablauf in Unity aussieht.

Zu einem bestimmten Zeitpunkt wird Unity allerdings den GUIManager erstellen. Der GUIManager ist bei uns die erste graphische Benutzeroberfläche die der Anwender zu Gesicht bekommt. Auf diesem führt der Host in diesem Szenario dann beispielhaft eine Wahl von Host und Fahrer aus. Nach getroffener Auswahl klickt dieser auf die Schaltfläche „Verbinden“. Durch das Betätigen der Schaltfläche wird der NetworkManager angesprochen, welcher auch von Unity im Vorfeld erstellt wurde. Der NetworkManager übernimmt hierbei die Logik den Server zu starten und Verbindungen entgegenzunehmen.

Nach erfolgreichem Verbinden, beschließt der Host in diesem Szenario das Programm zu beenden. Dazu betätigt er in der übergeordneten Unity Instanz eine Beenden-Schaltfläche. Unity beendet nun kaskadierend alle erzeugten Komponenten. Schließlich beendet Unity sich selbst und das Programm ist damit geschlossen.

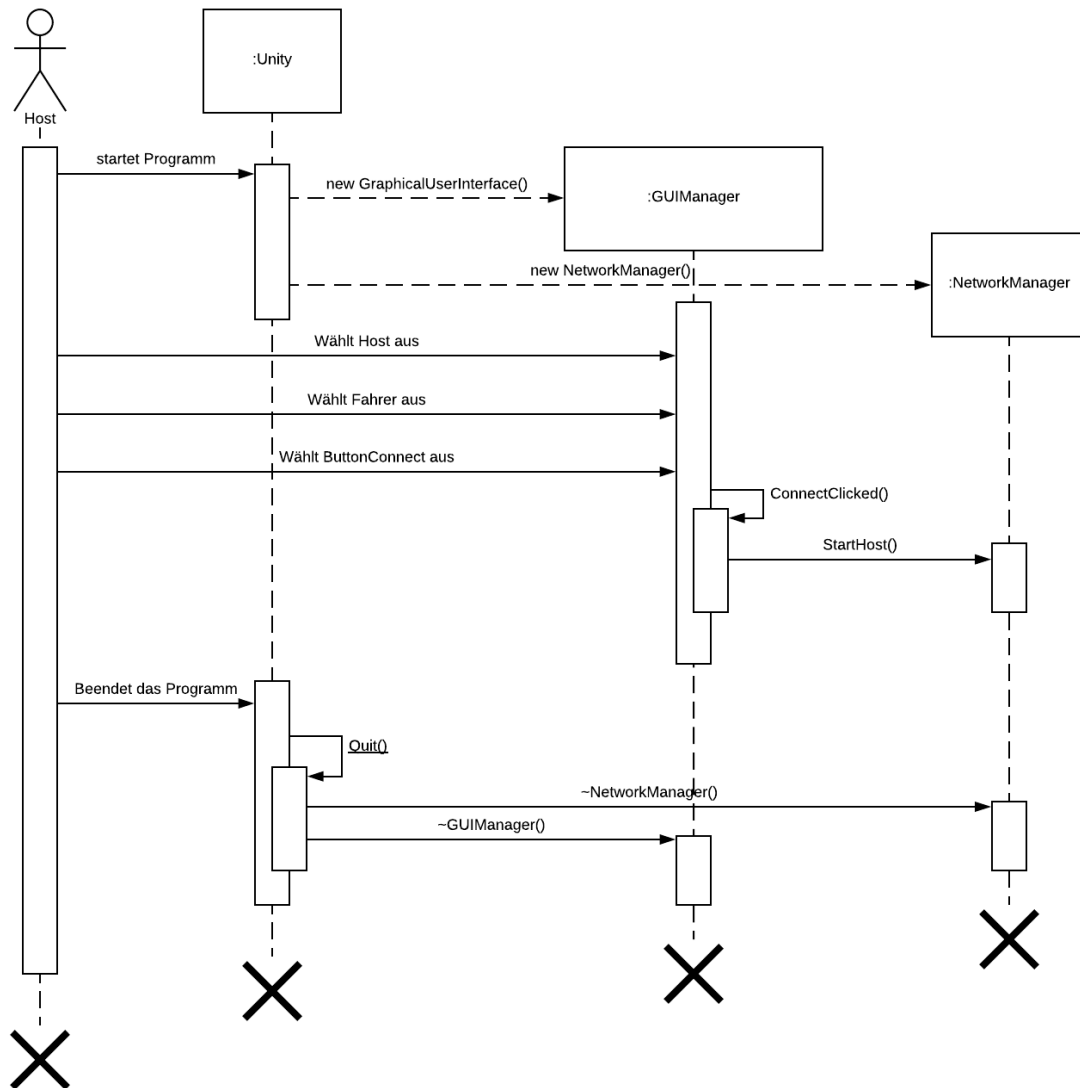


Abbildung 8: Sequenz-Diagramm 1 : Host erstellt Server

7.1.2 Gast startet das Programm und tätigt eine falsche Eingabe, welche danach korrigiert wird.

Dieses Sequenzdiagramm zeigt, wie ein Benutzer als Gast versucht mit einem Host Server zu verbinden und dabei falsche Verbindungsdaten eingibt. Er korrigiert diese Daten und verbindet sich beim zweiten Versuch korrekt.

Nach dem Starten des Programmes, erzeugt Unity die für die Simulation wichtigen Komponenten. In diesem Diagramm dargestellt wird der GUIManager und der NetzwerkManager. Der Benutzer möchte sich als Gast verbinden. Auf der graphischen Benutzeroberfläche ist bereits standardmäßig Gast ausgewählt. Daher gibt der Benutzer nun die IP-Adresse und den gewünschten Netzwerk Port ein. Außerdem möchte der Benutzer die Simulation als Fußgänger betreten. Auch dieser ist bereits standardmäßig ausgewählt. Nun wählt der Benutzer den ButtonConnect aus und der GUIManager gibt dem NetzwerkManager über StartClient() bescheid, dass er den Client mit der eingegebenen IP-Adresse und Netzwerk Port verbinden soll. Nach einer voreingestellten Zeit, prüft der GUIManager über isClientConnected(), ob der Client erfolgreich verbunden wurde. In diesem Szenario ist das allerdings nicht der Fall. Daher wirft der GUIManager eine Fehlermeldung um den Benutzer zu informieren, das die eingegebenen Daten nicht korrekt sind.

Dieser gibt jetzt neue Verbindungsdaten an und versucht sich erneut zu verbinden. Dieses mal sind die Eingaben korrekt und der Benutzer beendet anschließend das Programm.

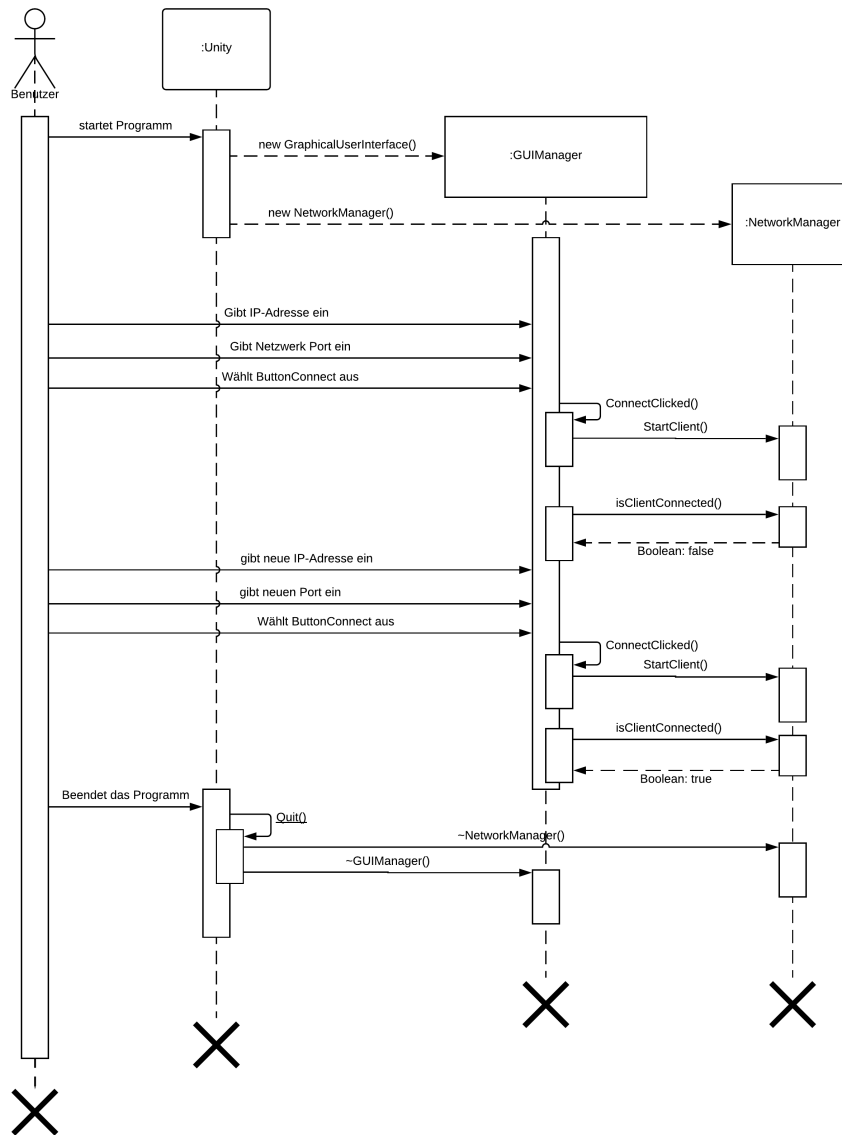


Abbildung 9: Sequenz-Diagramm 2 : Gast startet die Simulation und verbindet sich falsch

8 Anhang

